

CT520 – Algoritmos e Estruturas de Dados

Curso EN-AEL – 5º Ano,
2º Semestre

Docente: Prof. Cortez e Correia

cortez.correia@marinha.pt

Árvores.

Objetivos de aprendizagem

- Fornecer conhecimentos para compreender:
 - estruturas de dados não lineares: árvores

Sumário

Árvores Binárias (*Binary Trees*):

1. Binary Search Tree
2. AVL Tree
3. RedBlack Tree

Operações:

- a. Inserir
- b. Remover

Árvores N-árias (*Nary Trees*):

1. B Tree

Operações:

- a. Inserir
- b. Remover

Algoritmos de Ordenação (*Sort Algorithms*)

- Complexidade Algorítmica

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
<u>Quicksort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(\log(n))$
<u>Mergesort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Timsort</u>	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Heapsort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(1)$
<u>Bubble Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Insertion Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Selection Sort</u>	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Tree Sort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(n)$
<u>Shell Sort</u>	$\Omega(n \log(n))$	$\Theta(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
<u>Bucket Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n^2)$	$O(n)$
<u>Radix Sort</u>	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$	$O(n+k)$
<u>Counting Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n+k)$	$O(k)$
<u>Cubesort</u>	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$

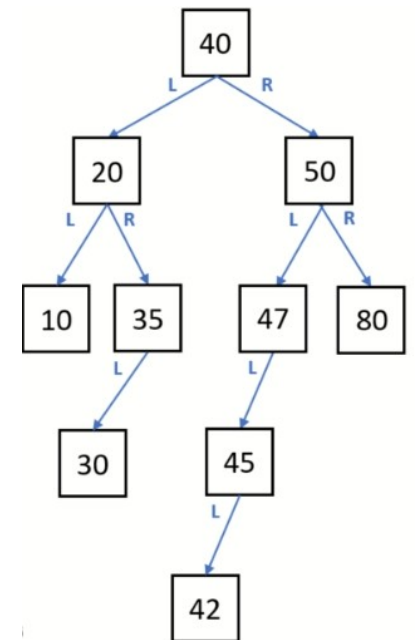
Árvores: Complexidade

- Complexidade Algorítmica

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
<u>Binary Search Tree</u>	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>Cartesian Tree</u>	N/A	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>B-Tree</u>	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>Red-Black Tree</u>	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>Splay Tree</u>	N/A	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	N/A	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>AVL Tree</u>	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>KD Tree</u>	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

Árvore Binária de Procura (*Binary Search Tree*)

- É uma árvore com um **nó raiz no topo**, a partir do qual saem ramos até às folhas da árvore. Cada **nó tem, no máximo, 2 filhos** (daí ser uma árvore binária)
- A BST tem a **propriedade de sequência** (*order property*), para cada nó:
 - Os valores da sub-árvore à esquerda do nó são menores
 - Os valores da sub-árvore à direita do nó são maiores
- Operações (preservando sempre as propriedades da BST):
 - Inserir (*Insert*)
 - Remover (*Delete*)
 - Procurar (*Search*)

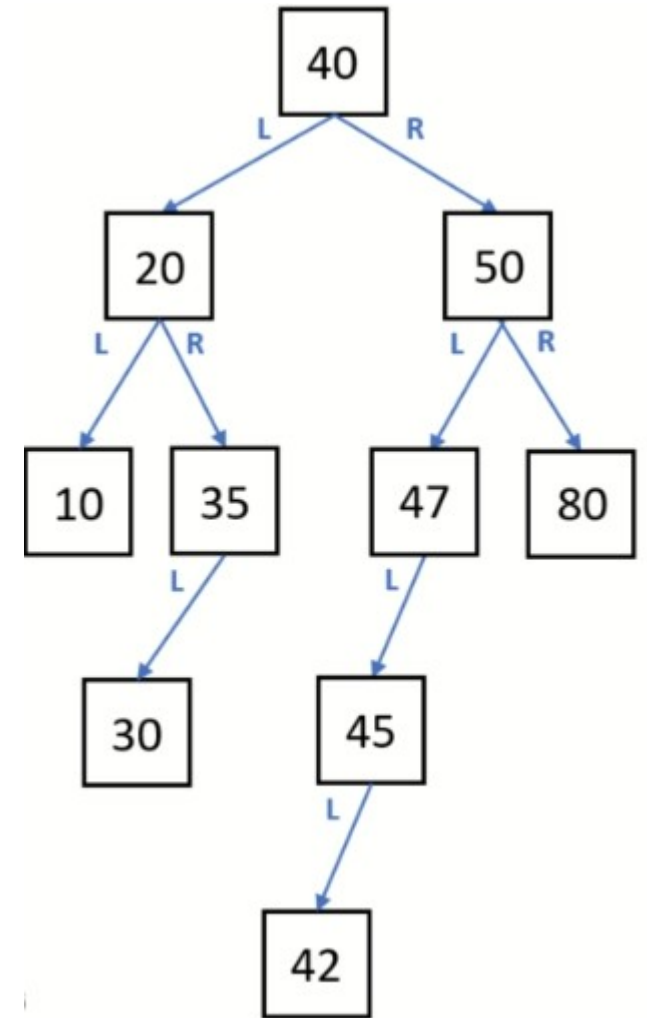


Binary Search Tree

- Uma percurso *in-order* de uma BST produz uma lista ordenada.
 - Percurso *in-order*: (Esquerda) (Próprio) (Direita)

```
void InOrderTraversal(root)
{
    InOrderTraversal(root->left);
    Print(root);
    InOrderTraversal(root->right);
}
```

- Esta característica torna a "Árvore Binária de Procura" eficiente para a operação de Busca

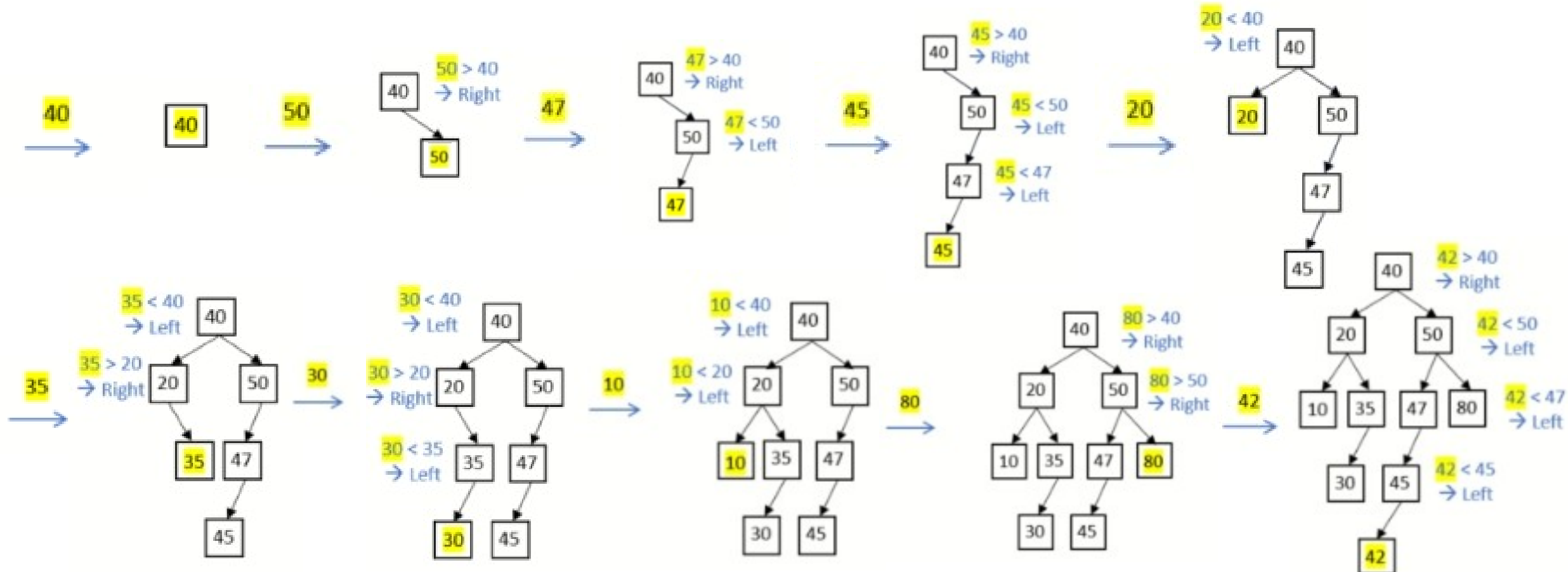


10, 20, 30, 35, (40), 42, 45, 47, 50, 80

Binary Search Tree

- Inserção de nós na BST

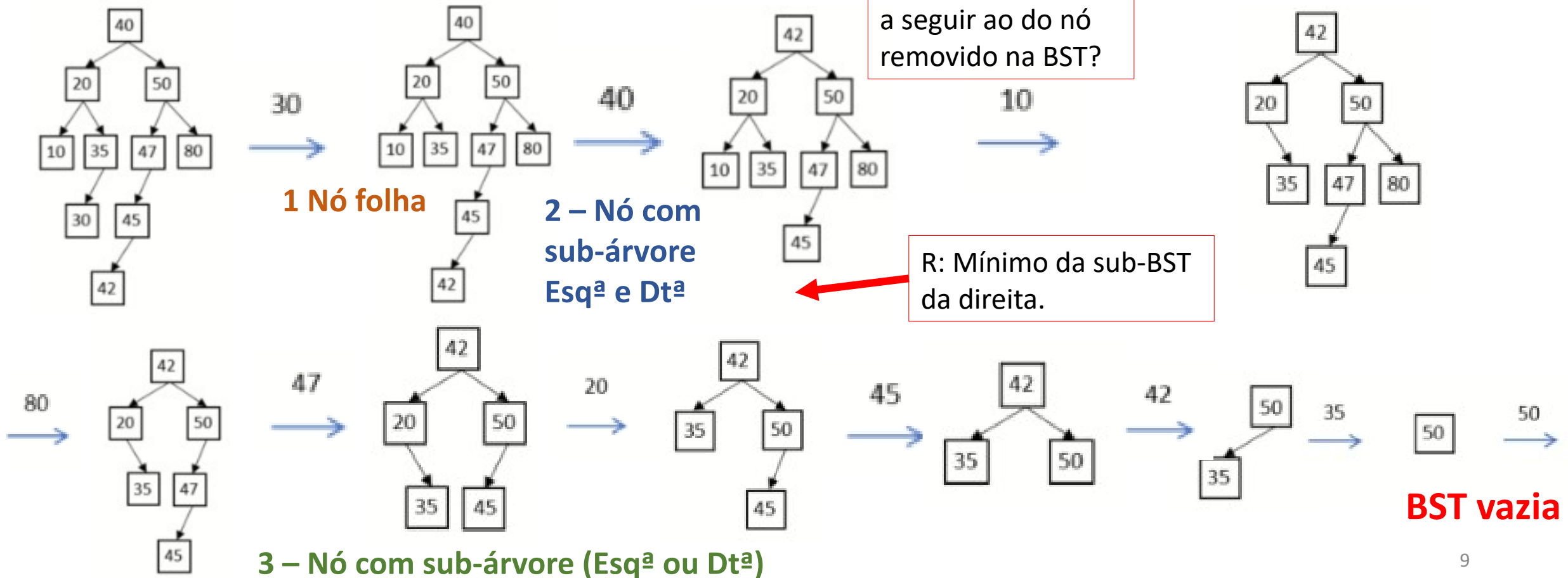
40, 50, 47, 45, 20, 35, 30, 10, 80, 42



Binary Search Tree

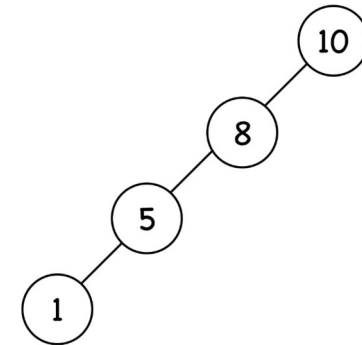
30, 40, 10, 80, 47, 20, 45, 42, 35, 50

- Remoção de nós na BST



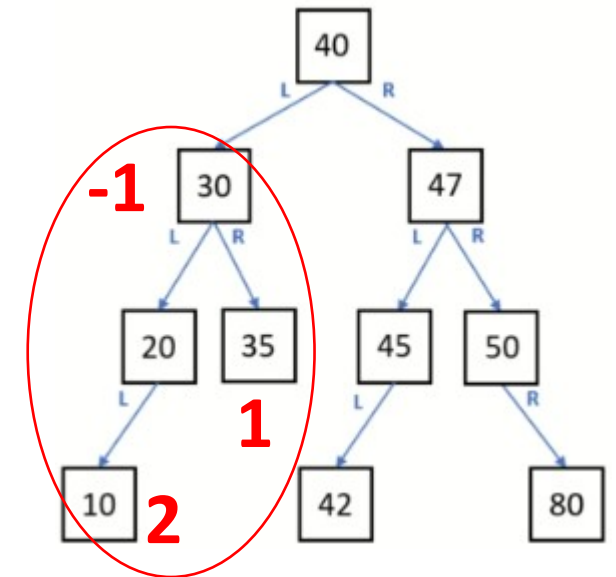
Binary Search Tree

- Problema da BST
 - Eficiência prejudicada se for criada a partir de nós com valores ordenados
 - Exemplo:
 - Criação de BST a partir dos valores: 10, 8, 5, 1
 - **Resultado:**
 - Complexidade temporal de uma lista: $O(n)$
- **Solução:** árvores balanceadas: complexidade temporal $O(\log(n))$
 - *AVL tree*
 - *Red-black tree*



Árvore AVL (*AVL Tree*)

- Árvore BST **auto-balanceada**: proposta por Adelson-Velsky e Landis (AVL)
- Possui as **propriedades da BST**: (i) cada nó tem, no máximo, 2 filhos; (ii) para dado nó, os valores dos nós à esq^a são menores e os dos nós à dt^a são maiores; (iii) o percurso *in-order* produz uma lista ordenada.
- É **balanceada** (a diferença, em valor absoluto, entre o nível (*level*) da sub-árvores da dt^a e da esq^a é menor ou igual a 1 ($-1 \leq \textit{balanceamento} \leq 1$))
- Operações (preservando as propriedades da árvore AVL):
 - Inserir (*Insert*)
 - Remover (*Delete*)
 - Procurar (*Search*)
 - **Nota**: as operações são semelhantes às da BST acrescidas de uma operação de balanceamento



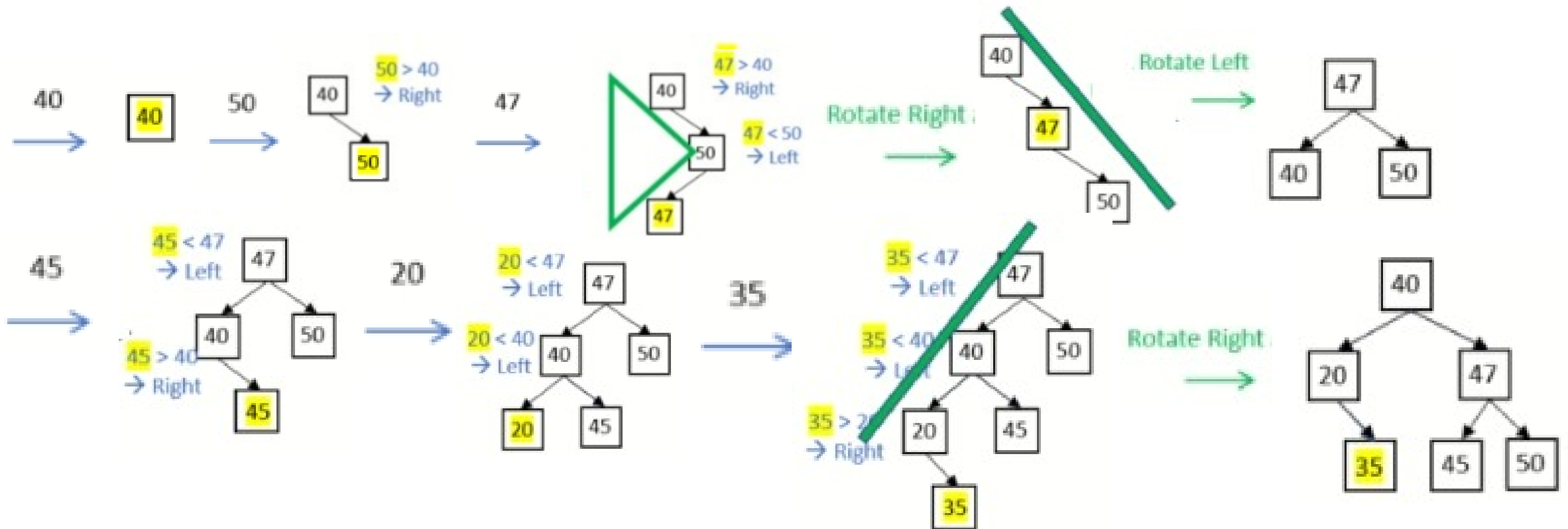
Árvore AVL (*AVL Tree*)

- Regras para o balanceamento
 - Verificar se a (sub)árvore com raiz no avô do nó inserido está balanceada
 - Se a (sub)árvore não está balanceada, havendo:
 1. Nós em linha para a esquerda (direita), rodar para a (sub)árvore para a direita (esquerda)
 2. Triângulo com um vértice apontado para a esquerda (direita) , rodar a (sub)árvore para a esquerda (direita) ficando os nós em linha. Aplicar de seguida a regra 1.

Árvore AVL (*AVL Tree*)

- Inserção de nós na árvore AVL

40, 50, 47, 45, 20, 35, 30, 10, 80, 42

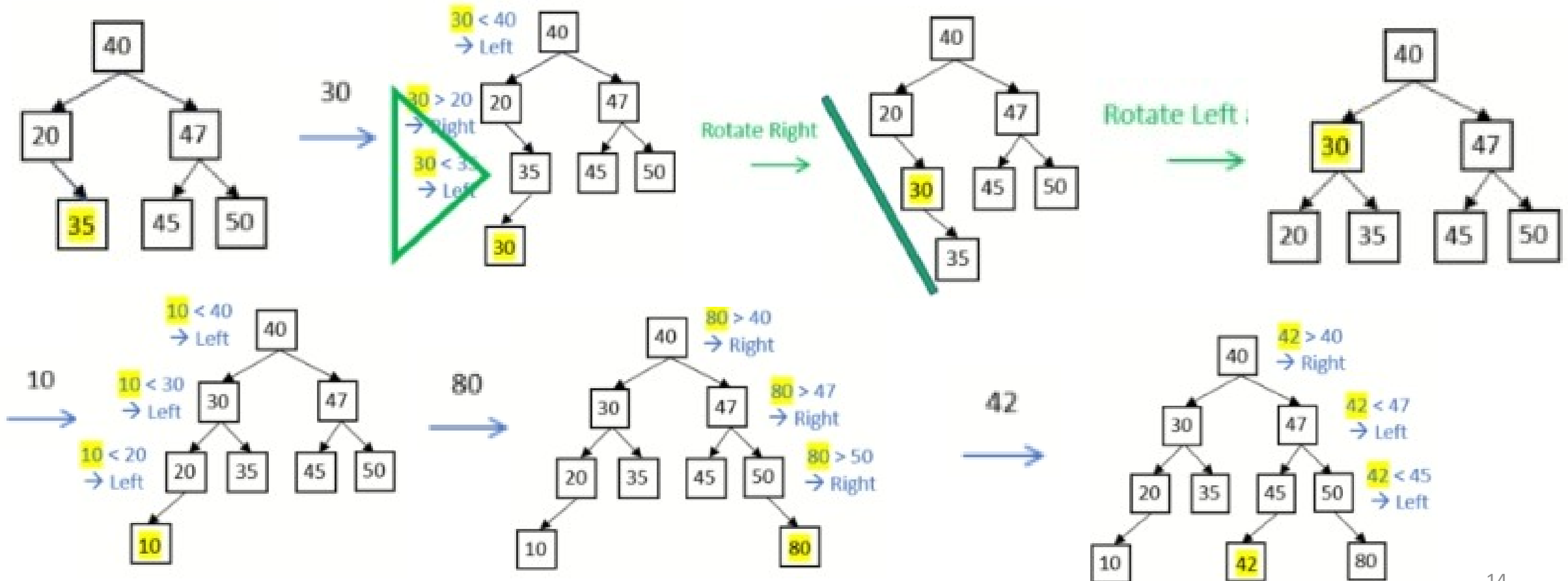


Nota: Após a inserção de cada nó, verificar a partir do nó avô, para cima, se a árvore se encontra balanceada.

Árvore AVL (*AVL Tree*)

- Inserção de nós na árvore AVL

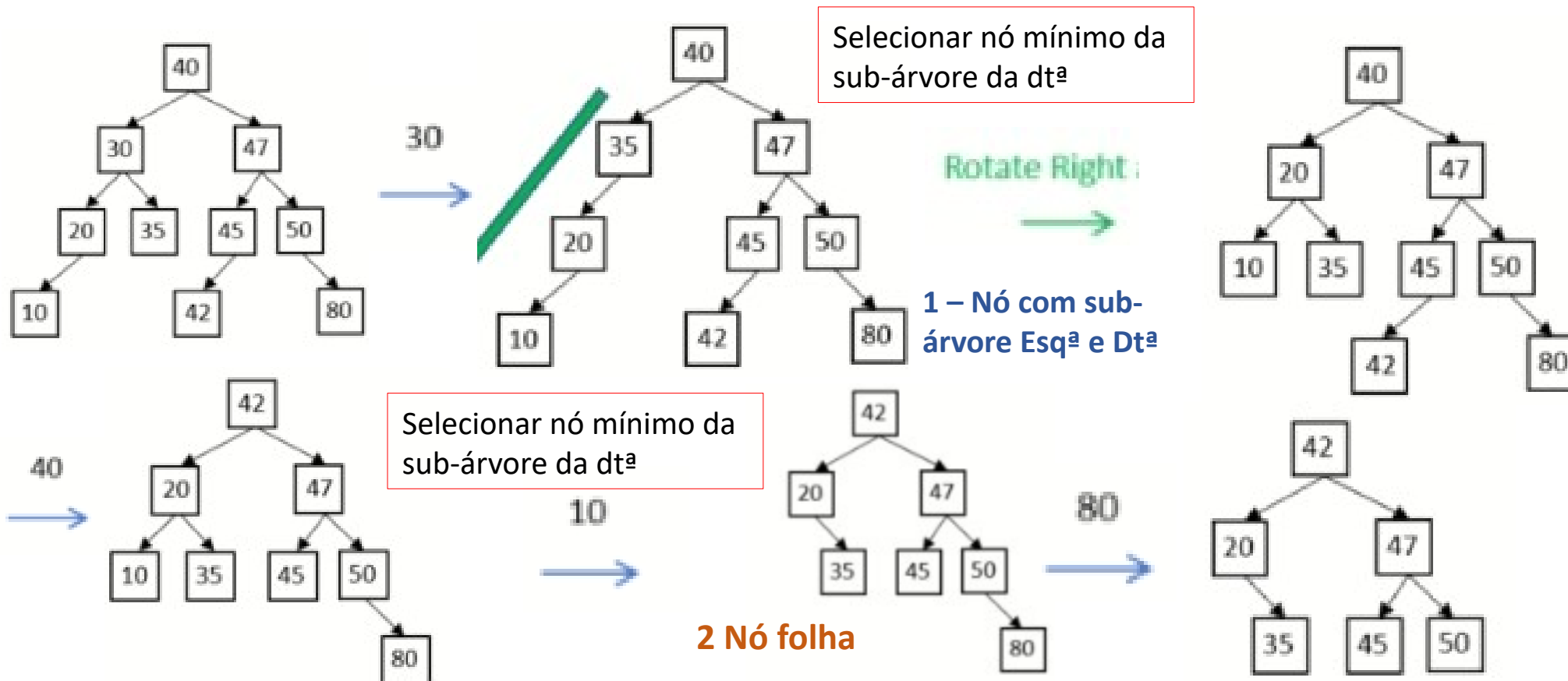
40, 50, 47, 45, 20, 35, 30, 10, 80, 42



Nota: Após a inserção de cada nó, verificar a partir do nó avô, para cima, se a árvore se encontra balanceada.

Árvore AVL (*AVL Tree*)

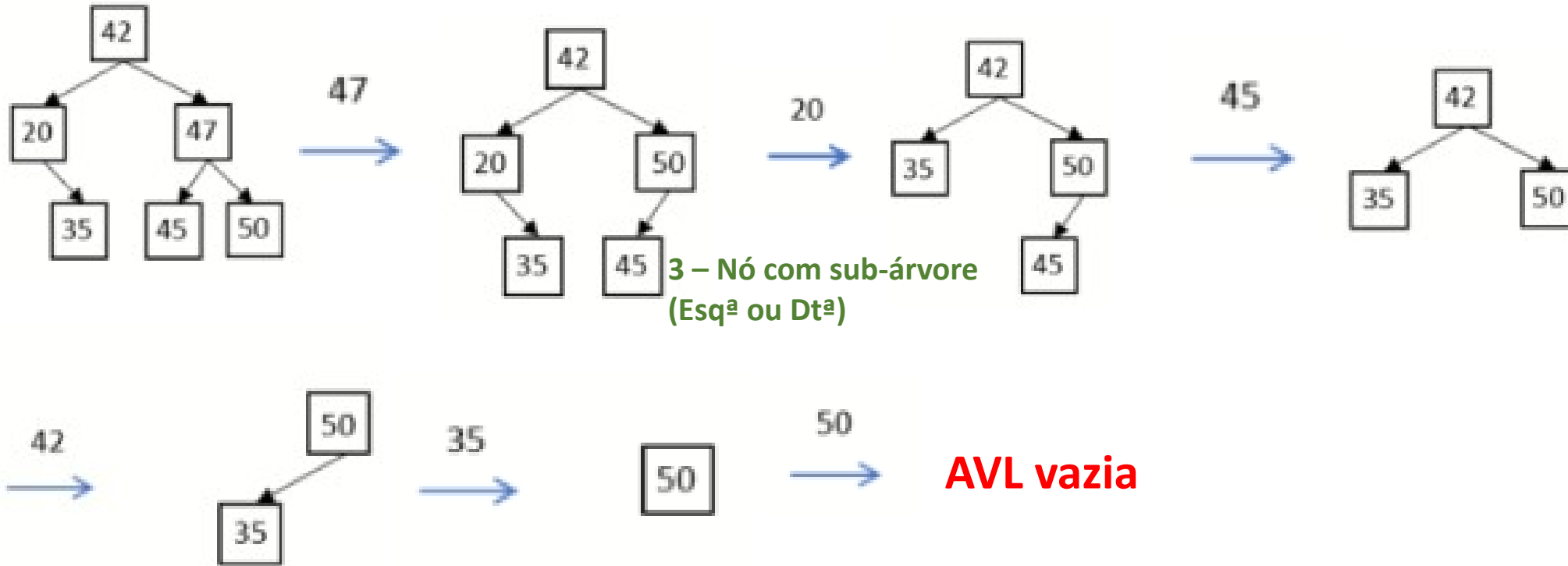
- Remoção de nós na árvore AVL 30, 40, 10, 80, 47, 20, 45, 42, 35, 50



Nota: Após a remoção de cada nó, verificar a partir do nó avô, para cima, se a árvore se encontra balanceada.

Árvore AVL (*AVL Tree*)

- Remoção de nós na árvore AVL 30, 40, 10, 80, 47, 20, 45, 42, 35, 50

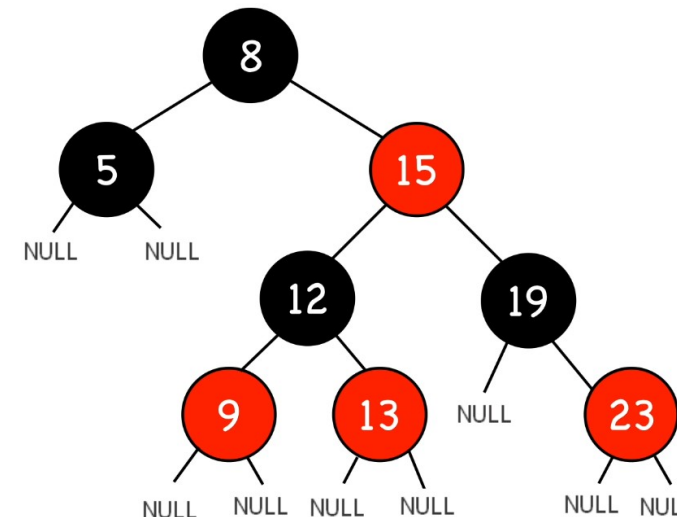


Nota: Após a remoção de cada nó, verificar a partir do nó avô, para cima, se a árvore se encontra balanceada.

Árvore Vermelha/Negra (**Red** **Black** Tree)

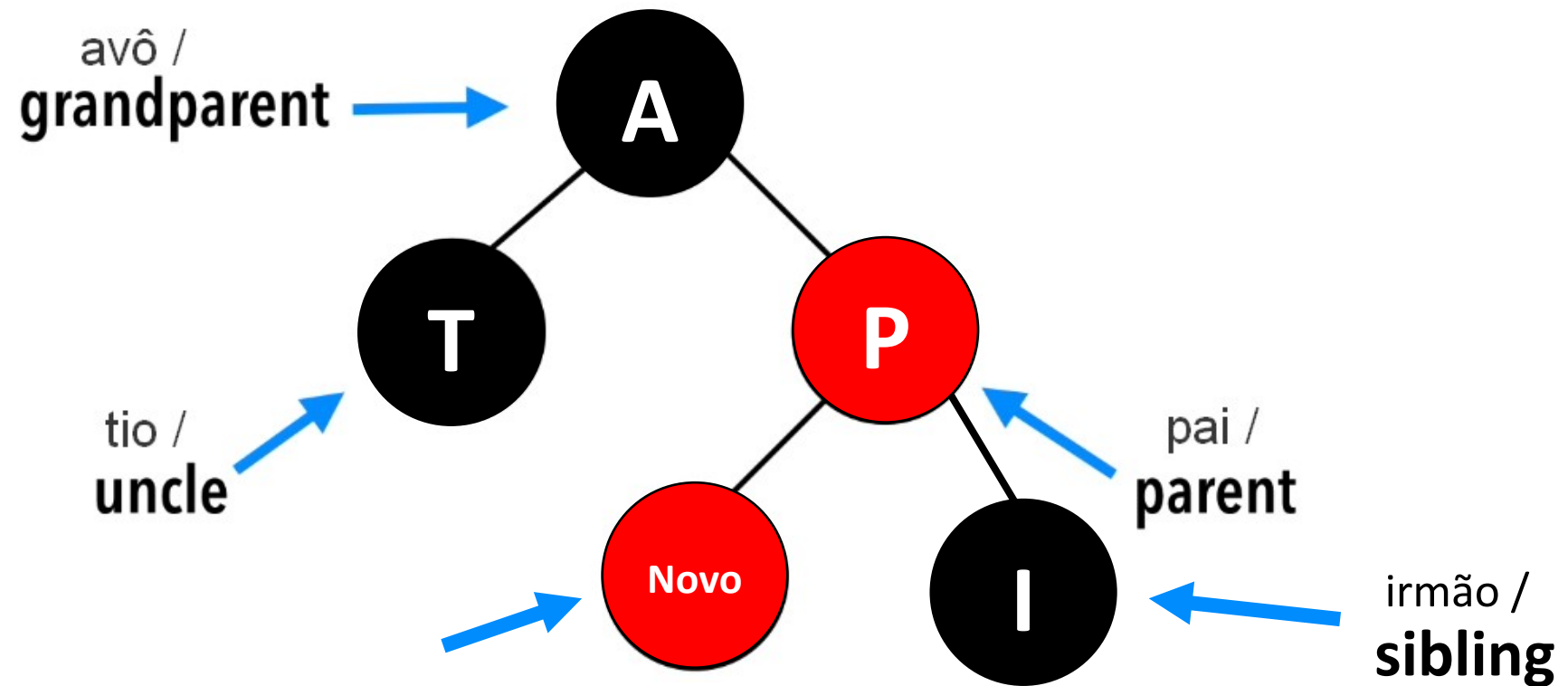
- Árvore BST **auto-balanceada**: proposta por Rudolf Bayer
- Possui as **propriedades da BST**: (i) cada nó tem, no máximo, 2 filhos; (ii) para dado nó, os valores dos nós à esq^a são menores e os dos nós à dt^a são maiores; (iii) o percurso *in-order* produz uma lista ordenada.
- É **balanceada** por **propriedades** das **RB** trees
- Operações (preservando as **propriedades** das **RB** trees):
 - Inserir (*Insert*)
 - Remover (*Delete*)
 - Procurar (*Search*)

Nota: as operações são semelhantes às da BST acrescidas da rotação para balanceamento



Red Black Tree

- Relações de parentesco de um novo nó

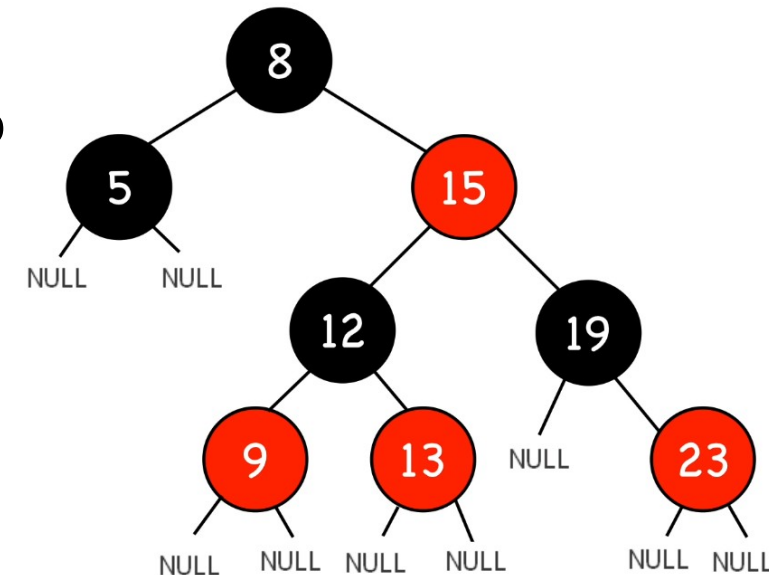


Um nó é sempre inserido como *red*

Red Black Tree

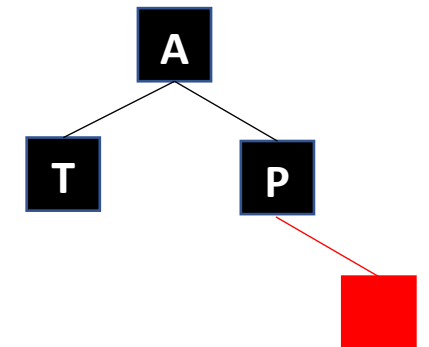
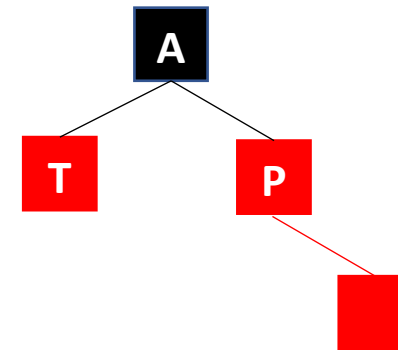
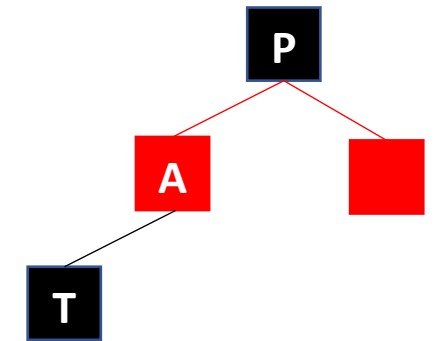
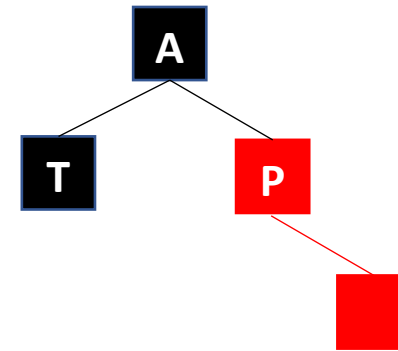
- **Propriedades** das **RB trees** que devem ser preservadas:

1. Um nó é vermelho (**red**) ou negro (**black**)
2. A raiz e os nós folha (NULL) são **black**
3. O pai de um nó **red** é **black**.
 - Não pode haver 2 nós consecutivos **red** (mas pode haver nós consecutivos **black**)
 - Os filhos esquerdo e direito de um nó **red** são **black**
4. Todos os caminhos de um nó, para os nós folha, têm o mesmo número de nós **black** (*black-height*)



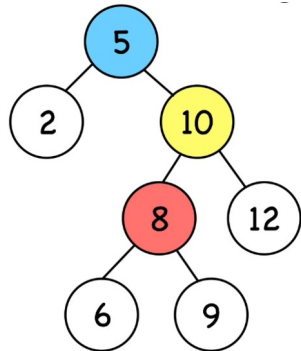
Red Black Tree

- Regras de balanceamento na inserção
 - Um nó é inserido como **red**
 - Se Tio **black** → Rodar e colorir
 - Se Tio **red** → Colorir

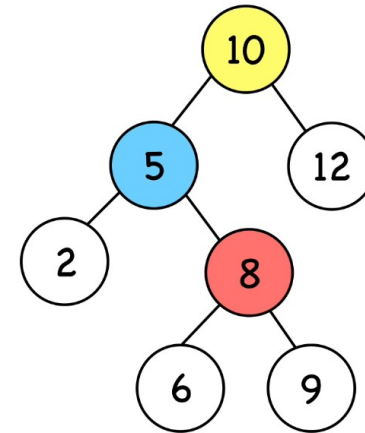


Red Black Tree

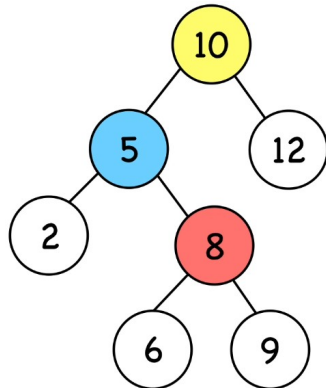
- Rotações para preservar as **propriedades** das **RB** trees – $O(1)$
 - Rotação à Esquerda (*left-rotate* LR)



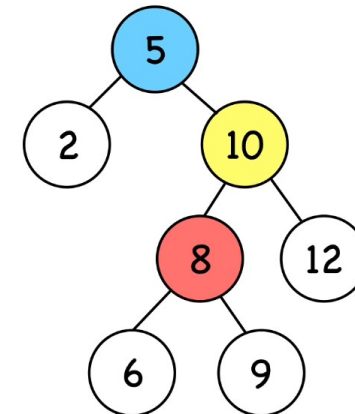
left-rotate(5)
→



- Rotação à Direita (*right-rotate* RR)



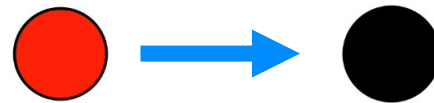
right-rotate(10)
→



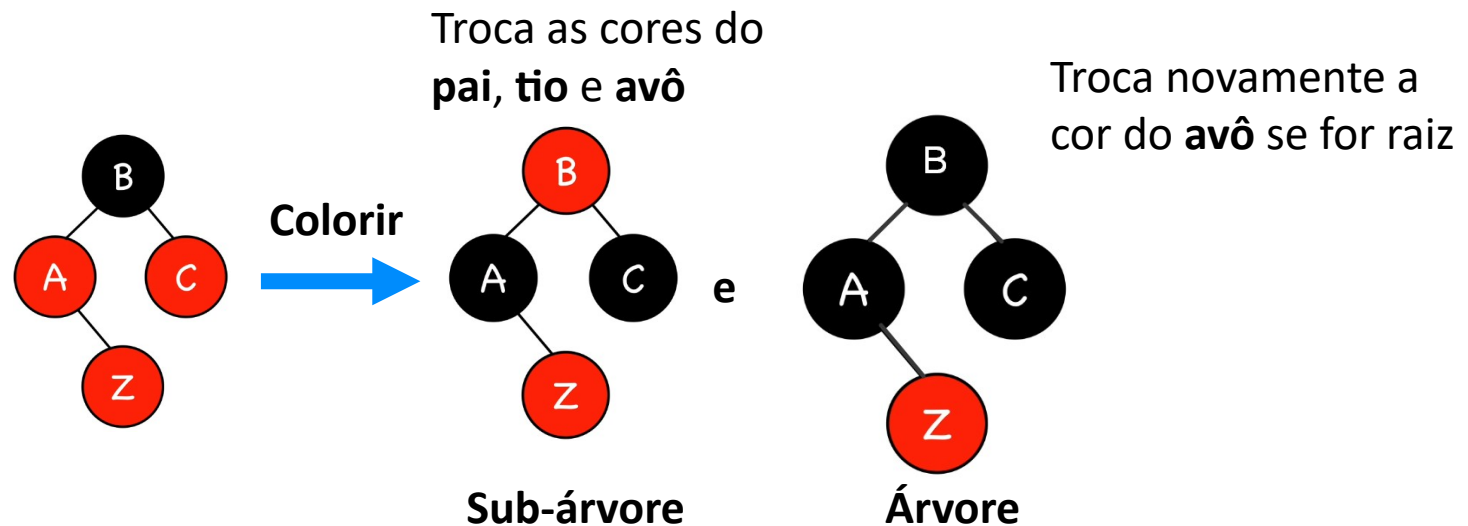
Red Black Tree

- Cenários possíveis de inserção de um novo nó:

1. Como raiz



2. Com tio *red*

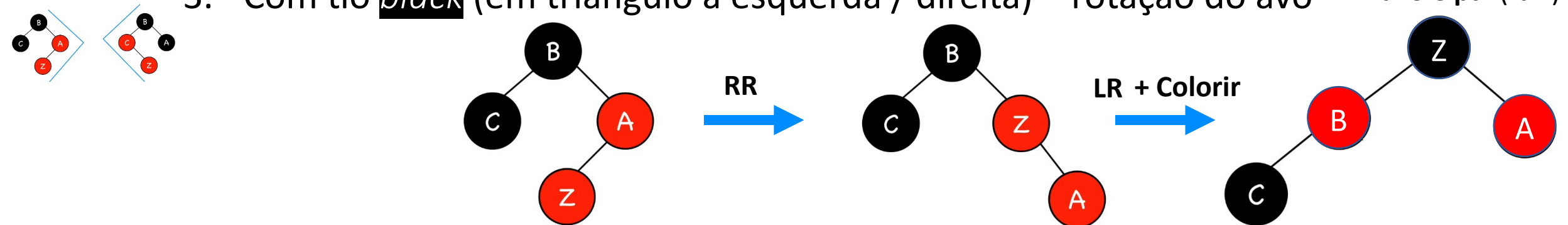


Red Black Tree

- Cenários possíveis de inserção de um novo nó:

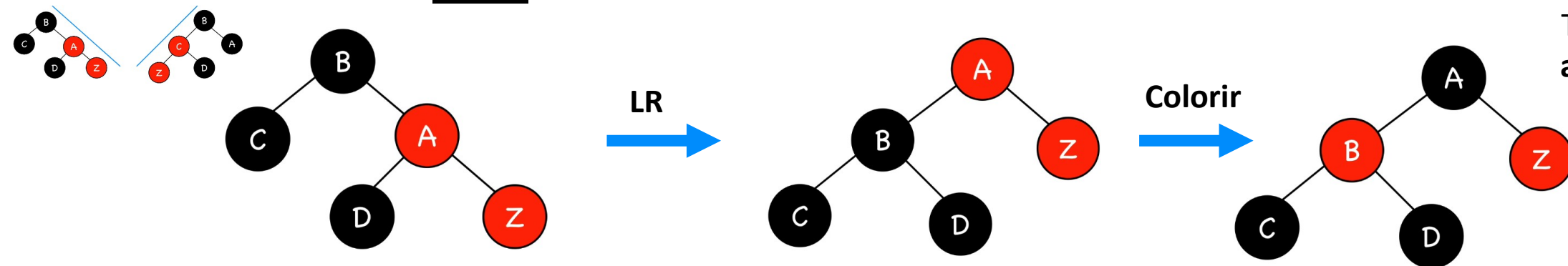
3. Com tio **black** (em triângulo à esquerda / direita) – rotação do avô

Troca a cor do avô e pai (raiz)



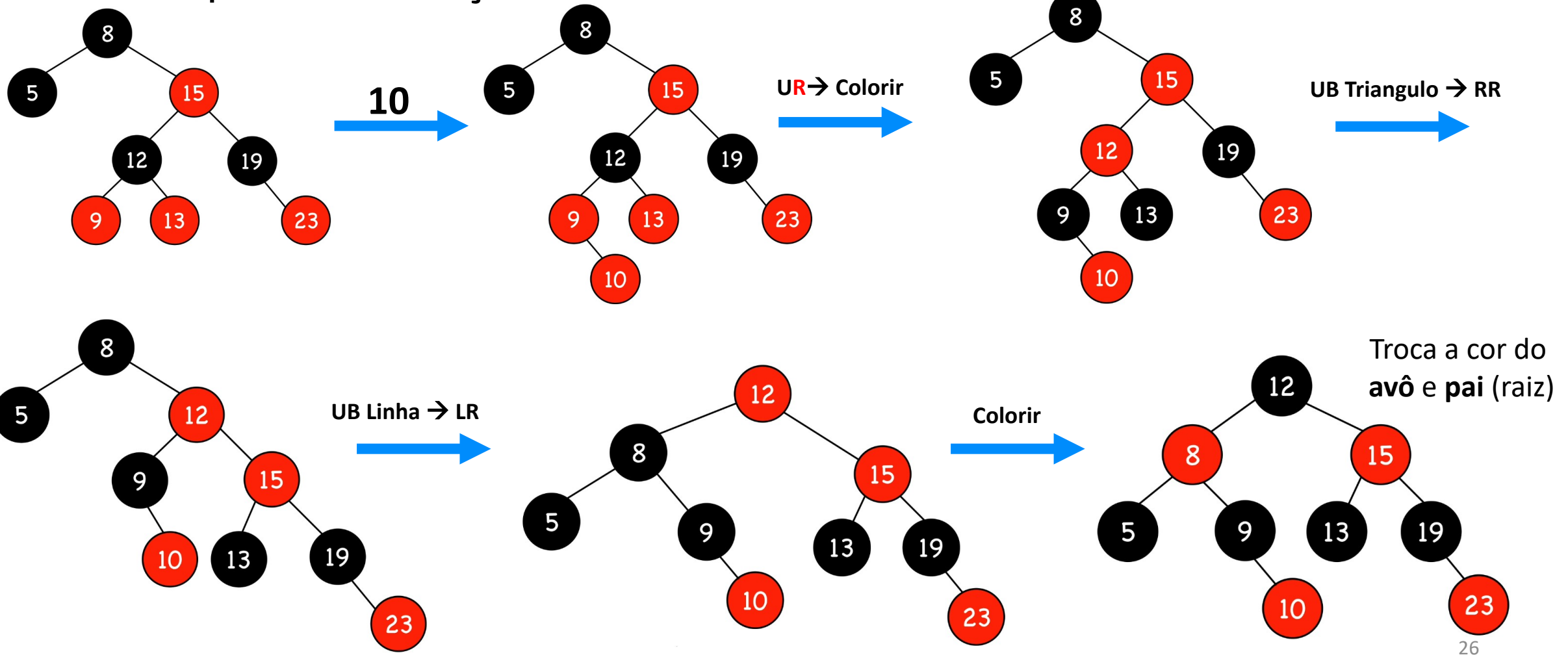
3. Com tio **black** (em linha) – rotação do avô

Troca a cor do avô e pai (raiz)



Red Black Tree

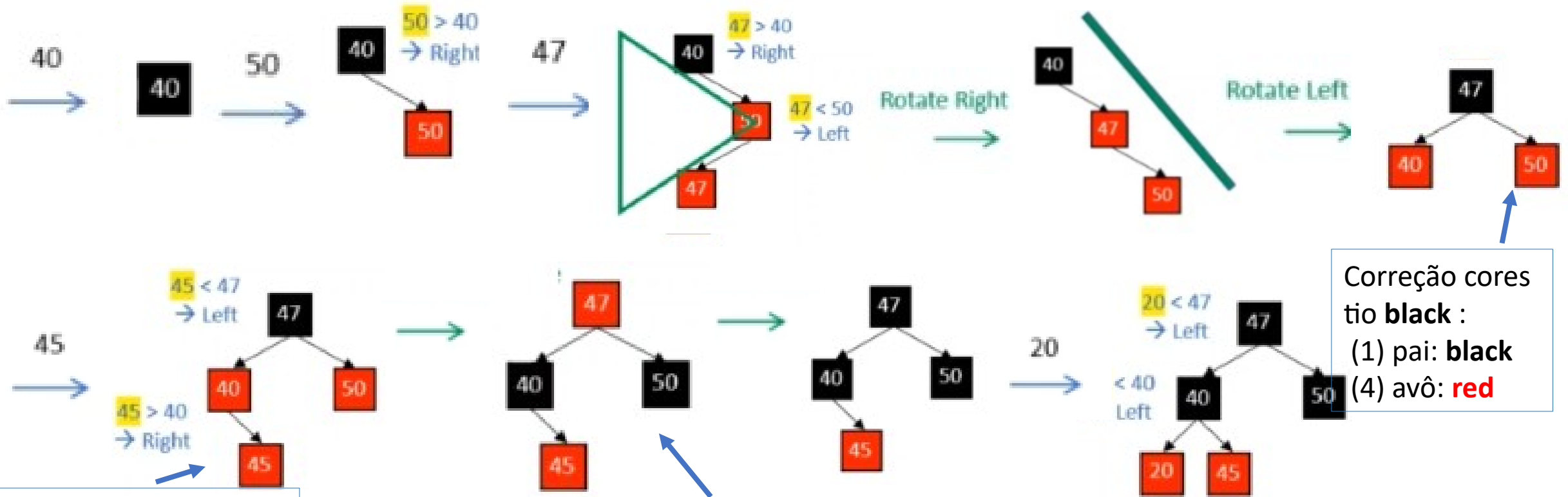
- Exemplos de inserção



Red Black Tree

- Inserção de nós na **RB** tree

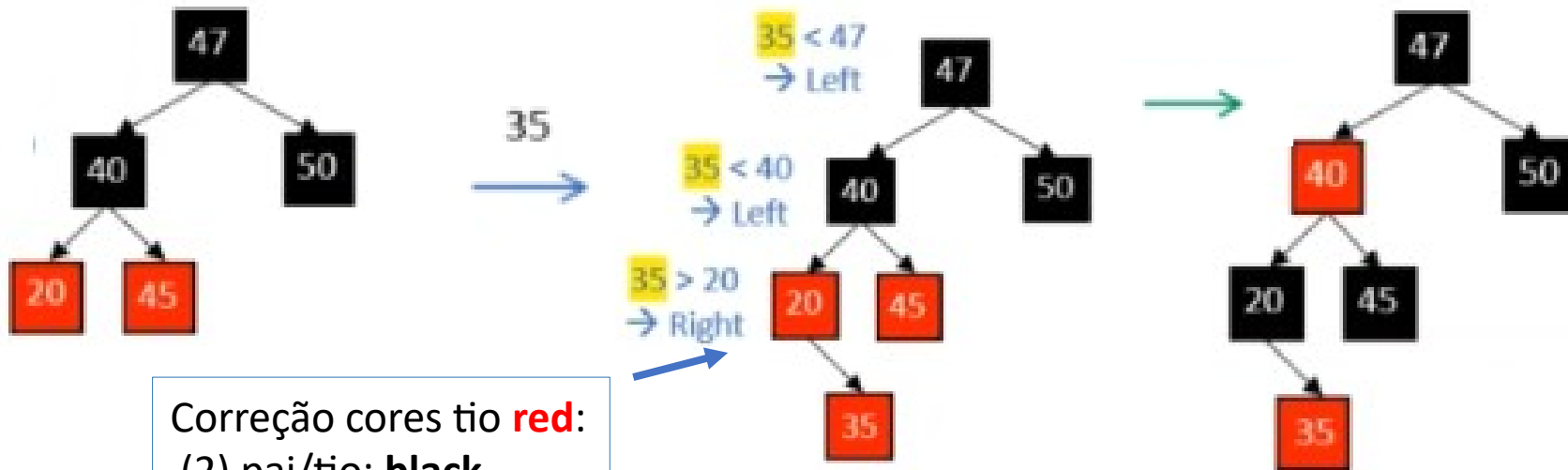
40, 50, 47, 45, 20, 35, 30, 10, 80, 42



Red Black Tree

- Inserção de nós na **RB** tree

40, 50, 47, 45, 20, 35, 30, 10, 80, 42

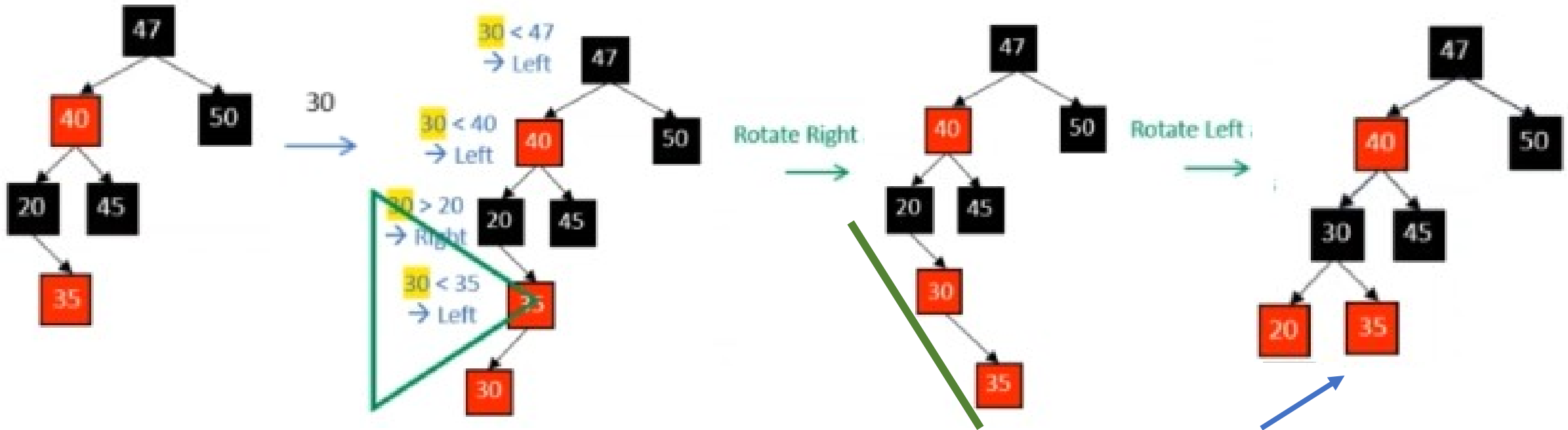


Correção cores tio **red**:
(2) pai/tio: **black**
(3) avô: **red**

Red Black Tree

- Inserção de nós na **RB tree**

40, 50, 47, 45, 20, 35, 30, 10, 80, 42

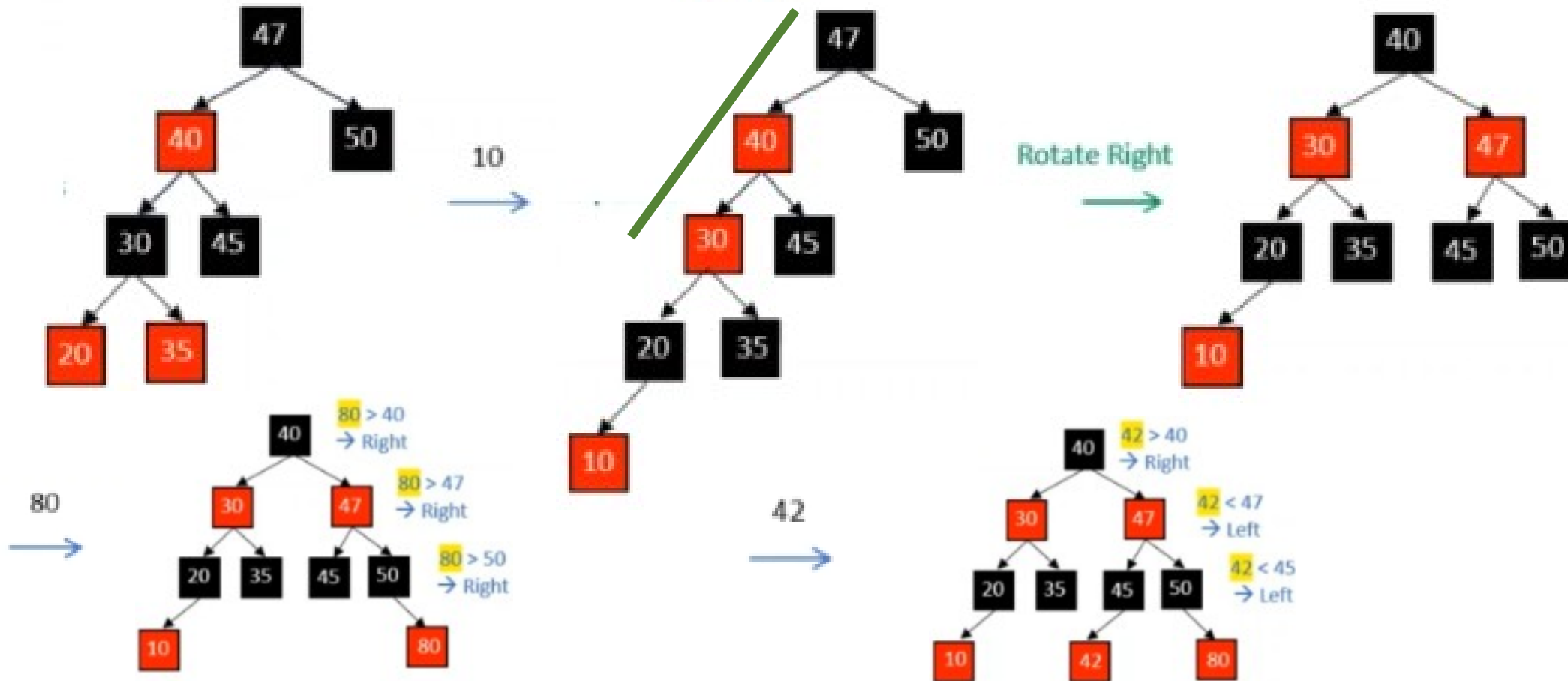


Correção cores tio **black** :
(1) pai: **black**
(4) avô: **red**

Red Black Tree

- Inserção de nós na **RB** tree

40, 50, 47, 45, 20, 35, 30, 10, 80, 42



Árvore B (*B-Tree*)

- Cada nó armazena um determinado número de pares *chave-valor*.
- É semelhante a uma árvore de pesquisa binária balanceada (BST), exceto na capacidade de um nó da árvore, que **não é limitada a apenas uma chave**, podendo ser maior.
- Uma *B-Tree* é definida pelo parâmetro D - **grau máximo de ramificação** (*maximum branching degree*):
 - o **número máximo de filhos** que qualquer nó da árvore pode ter.

B-Tree

For any node in a D - d B-Tree		Example : 2-3 B-tree	Example : 2-3-4 B-tree
Maximum children	D	$D = 3$	$D = 4$
Minimum children	$d = \left\lceil \frac{D}{2} \right\rceil$	$d = \left\lceil \frac{D}{2} \right\rceil = 2$	$d = \left\lceil \frac{D}{2} \right\rceil = 2$
Maximum keys	$K = D - 1$	$K = D - 1 = 2$	$K = D - 1 = 3$
Minimum keys	$k = d - 1$	$k = d - 1 = 1$	$k = d - 1 = 1$

- O número de chaves em um nó da árvore, a qualquer momento é:

$$k \leq \text{número de chaves por nó} \leq K$$

- Um nó de árvore com **m chaves** deve ter pelo menos **$m + 1$ filhos**, exceto para:
 - Nós de folha (têm sempre 0 filhos),
 - O nó raiz, quando nós suficientes não foram ainda criados.

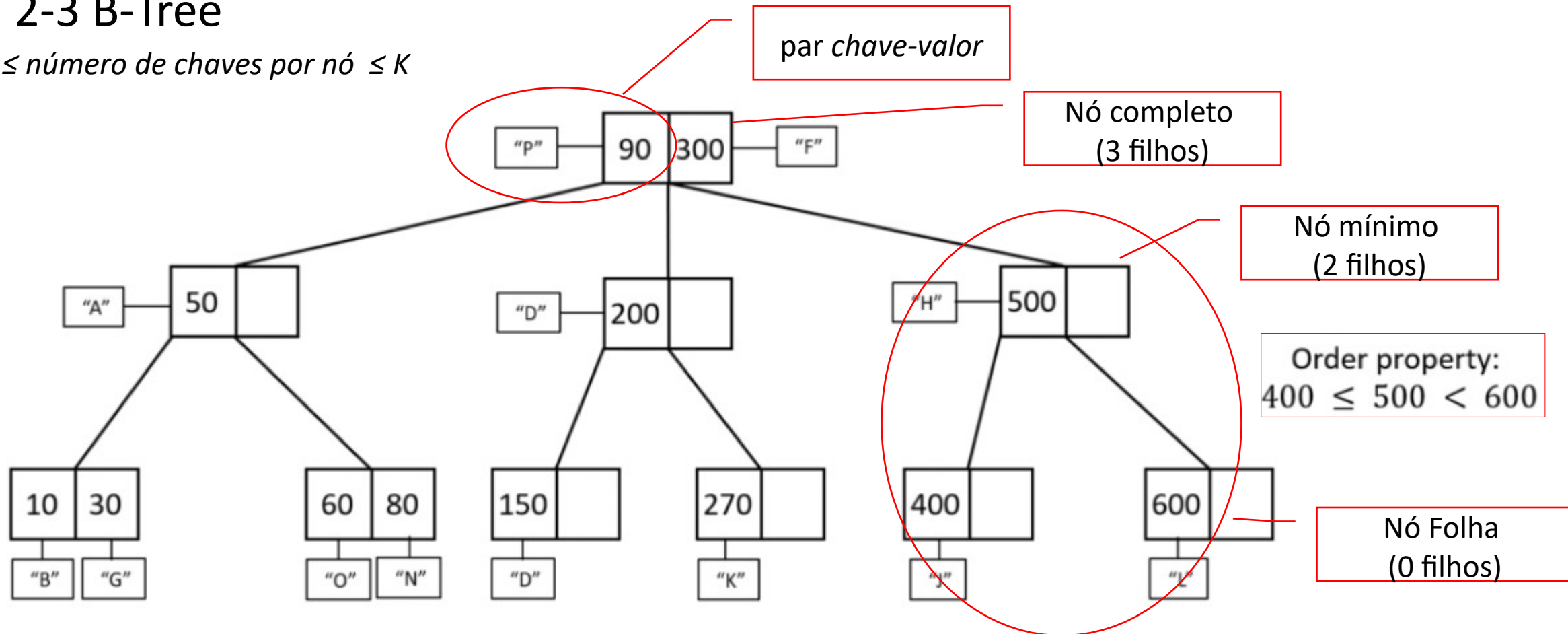
B-Tree

- Numa *B-Tree*, os nós internos e folhas **armazenam pares de chave-valor**.
 - É diferente das *B+Tree*, onde apenas os nós de folha podem armazenar chave-valor, com os nós internos atuando como um índice para as folhas
- Semelhante à **árvore de procura binária (BST)**, as *B-Tree* também têm a propriedade de sequência (*order property*):
 - Para determinada chave K num nó da árvore, todas as chaves na subárvore à esquerda dessa chave são menores que K, e todas as chaves na subárvore da direita são maiores que a chave K
- As operações comuns na *B-Tree* são (preservando sempre as propriedades da *B-Tree*):
 - Inserir
 - Remover
 - Procurar

B-Tree

- 2-3 B-Tree

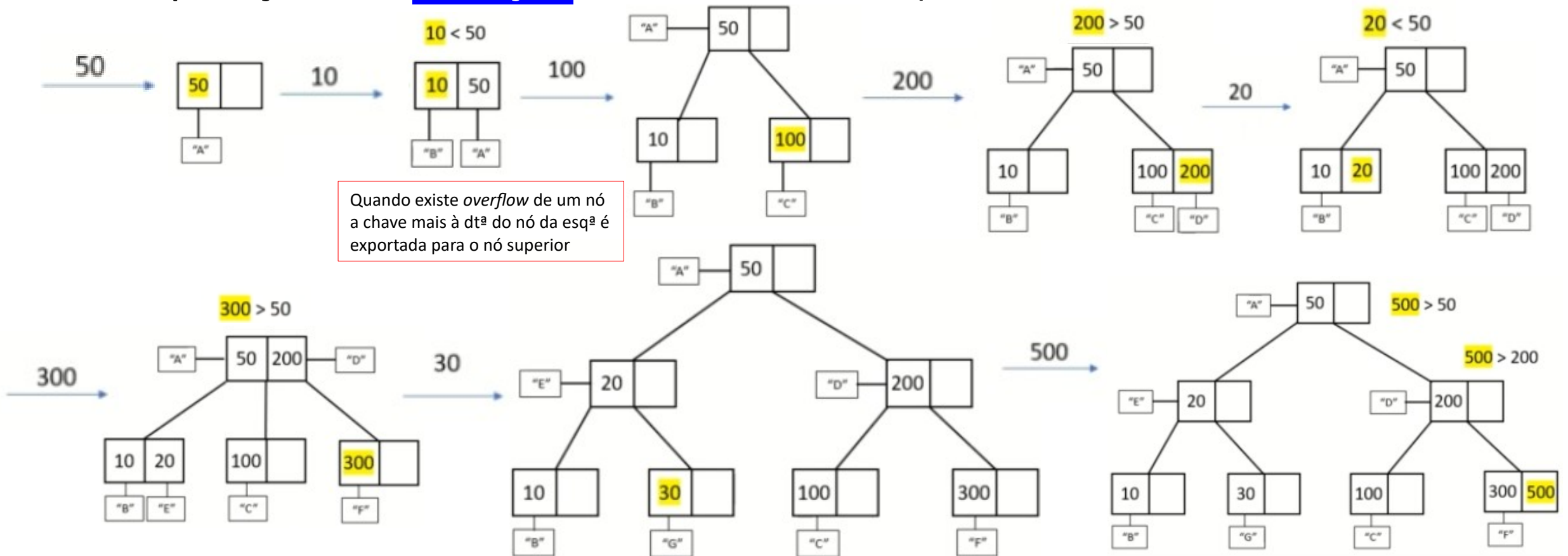
$k \leq \text{número de chaves por nó} \leq K$



B-Tree

50, 10, 100, 200, 20, 300, 30, 500, 250, 400, 270, 600, 150, 80, 60, 90

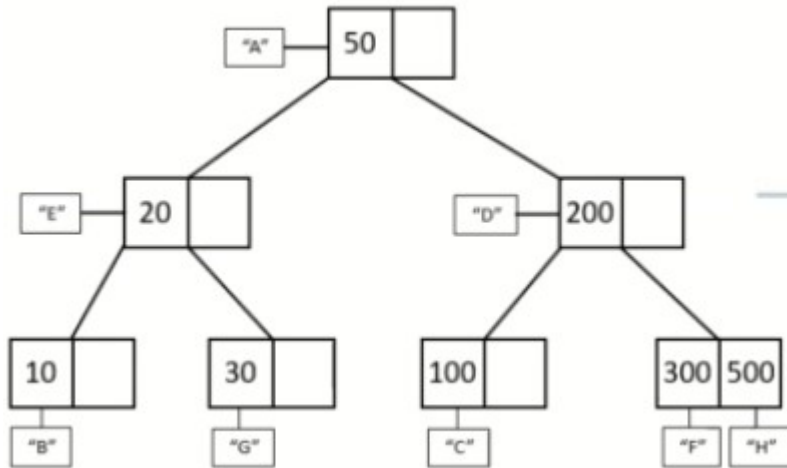
- Operações de **inserção** numa 2-3 B-Tree ($1 \leq n^{\circ} \text{ chaves no nó} \leq 2$)



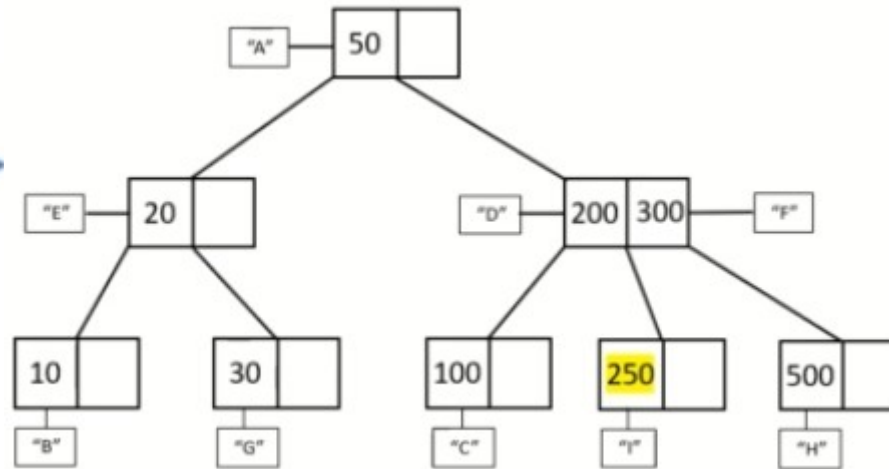
B-Tree

50, 10, 100, 200, 20, 300, 30, 500, 250, 400, 270, 600, 150, 80, 60, 90

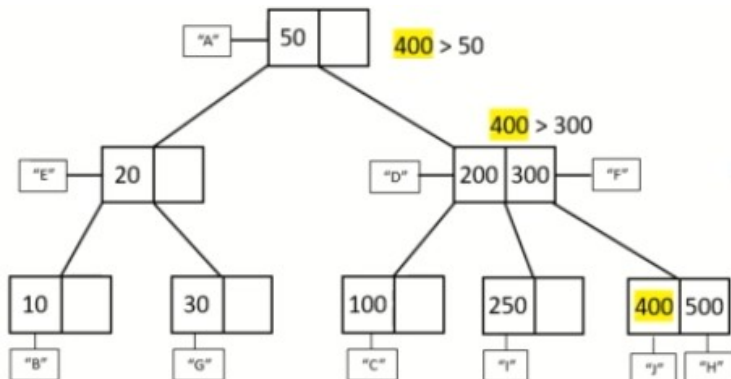
- Operações de **inserção** numa 2-3 B-Tree ($1 \leq n^o \text{ chaves no nó} \leq 2$)



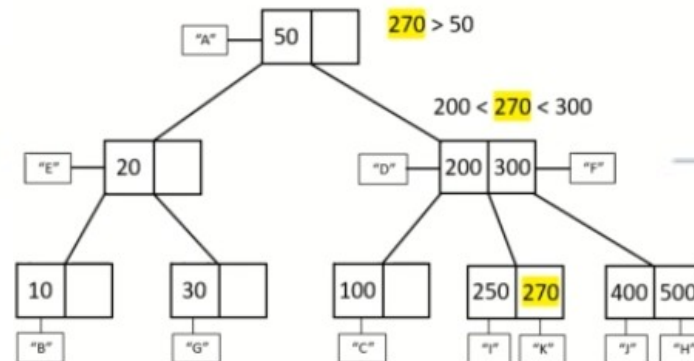
250



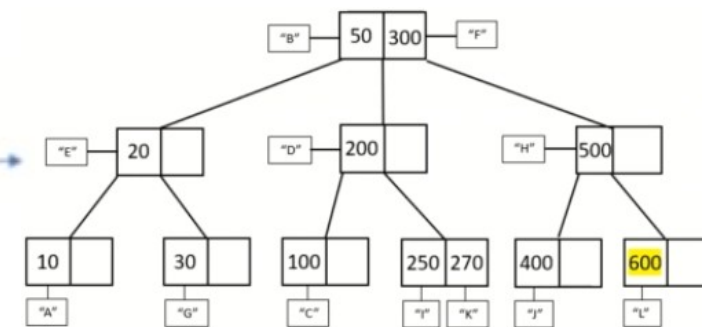
400



270



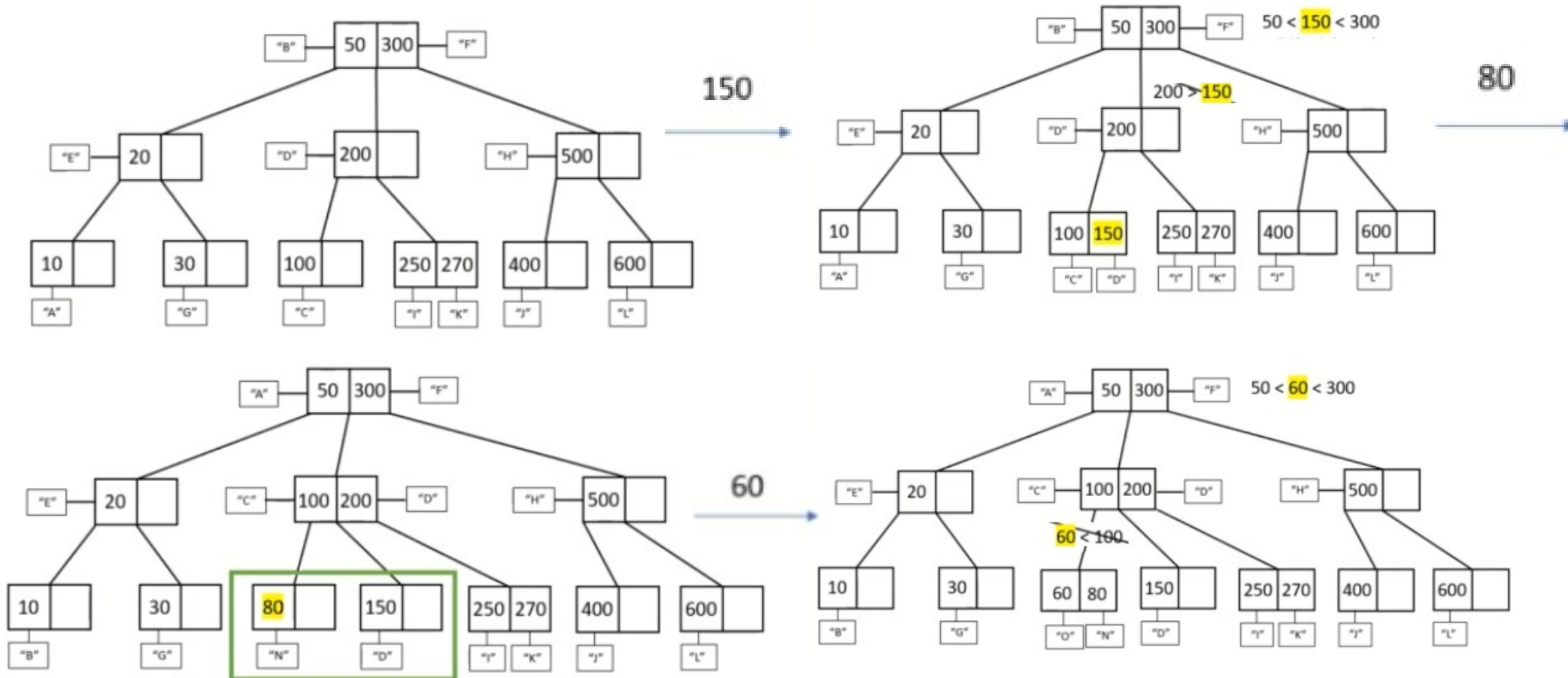
600



B-Tree

50, 10, 100, 200, 20, 300, 30, 500, 250, 400, 270, 600, 150, 80, 60, 90

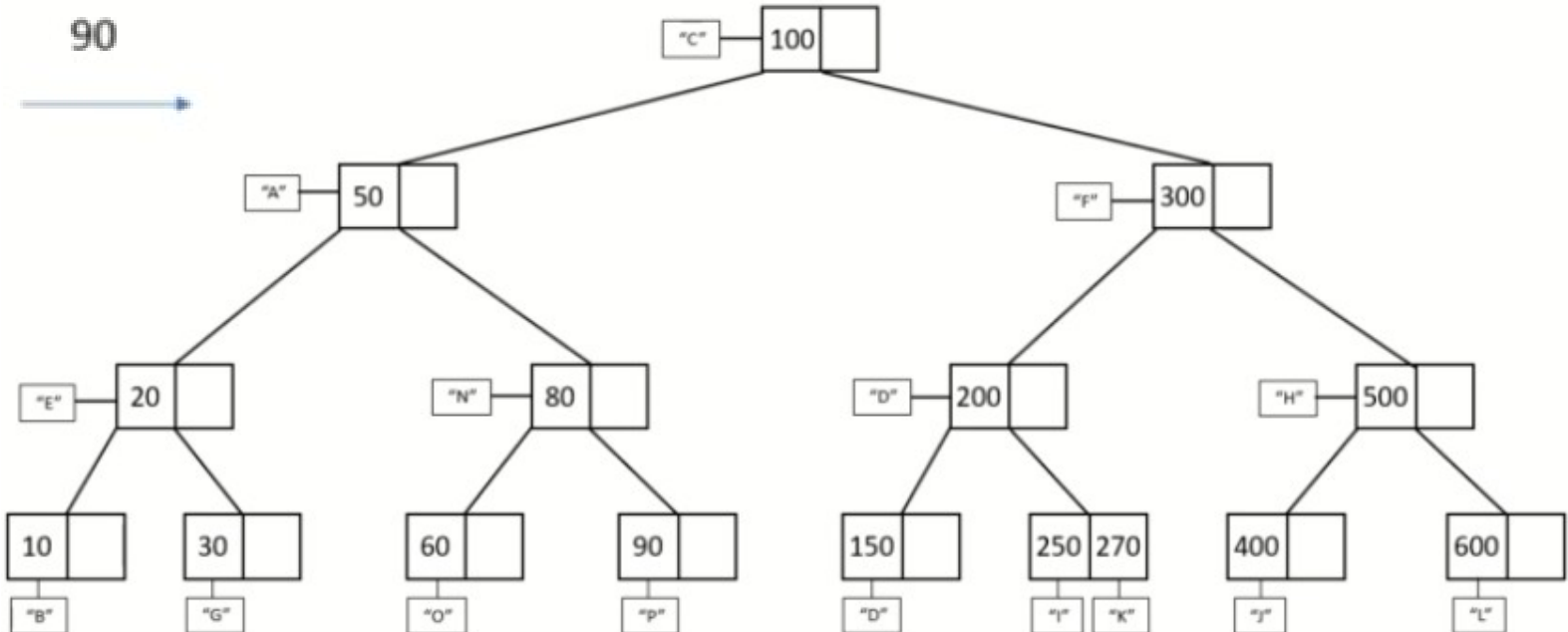
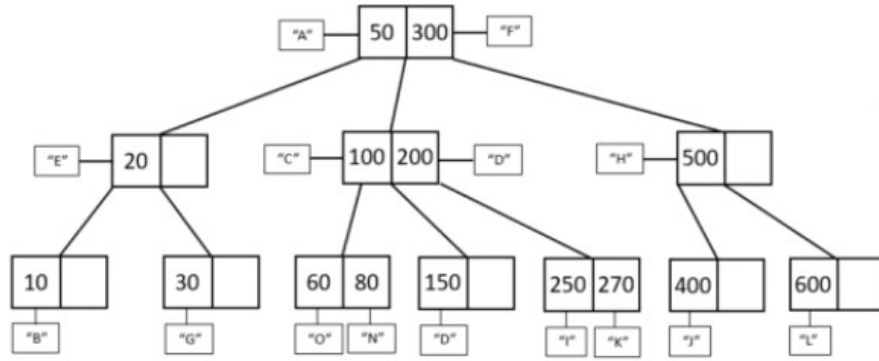
- Operações de **inserção** numa 2-3 B-Tree ($1 \leq n^o \text{ chaves no nó} \leq 2$)



B-Tree

50, 10, 100, 200, 20, 300, 30, 500, 250, 400, 270, 600, 150, 80, 60, 90

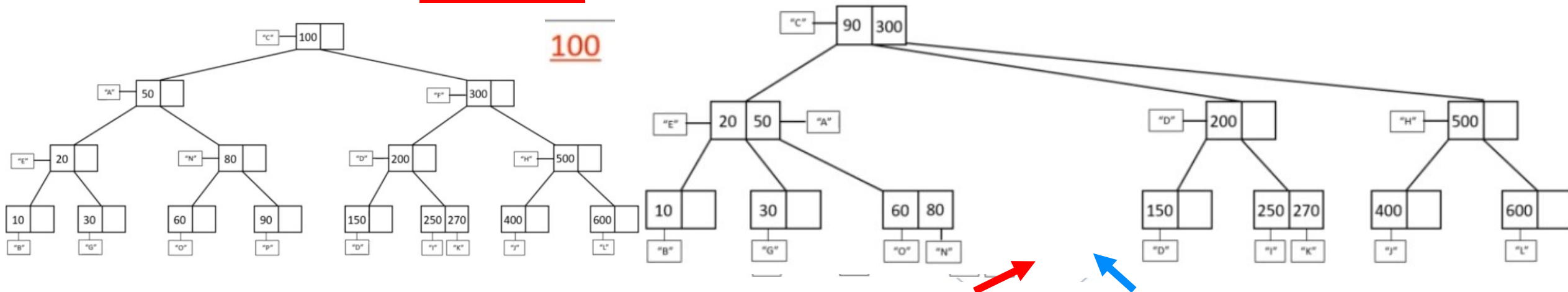
- Operações de **inserção** numa 2-3 B-Tree ($1 \leq n^o \text{ chaves no nó} \leq 2$)



B-Tree

100, 20, 250, 600, 270, 60, 10, 300, 80, 150, 400, 30, 90, 200, 50, 500

- Operações de **remoção** numa 2-3 B-Tree ($1 \leq n^{\circ} \text{ chaves no nó} \leq 2$)



Substituir o valor do nó a remover pelo valor máximo da subárvore da esquerda (valor imediatamente anterior)

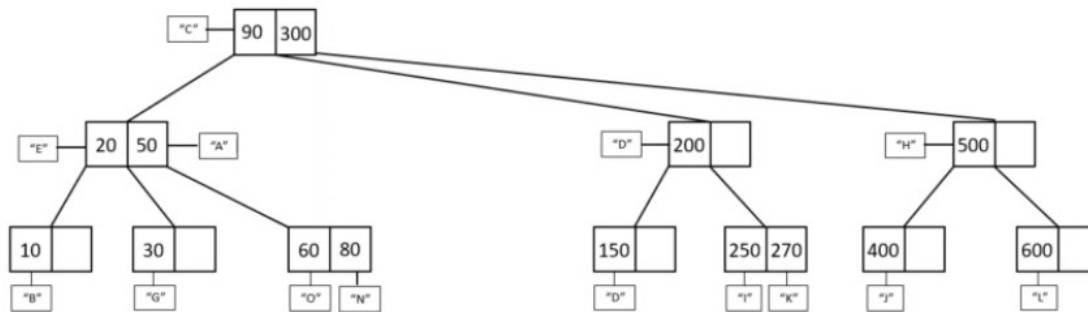
Como o nó ficou vazio encontrar um irmão com pelo menos $k + 1$ chaves

Caso não haja procurar um irmão com k chaves para fazer junção de nós

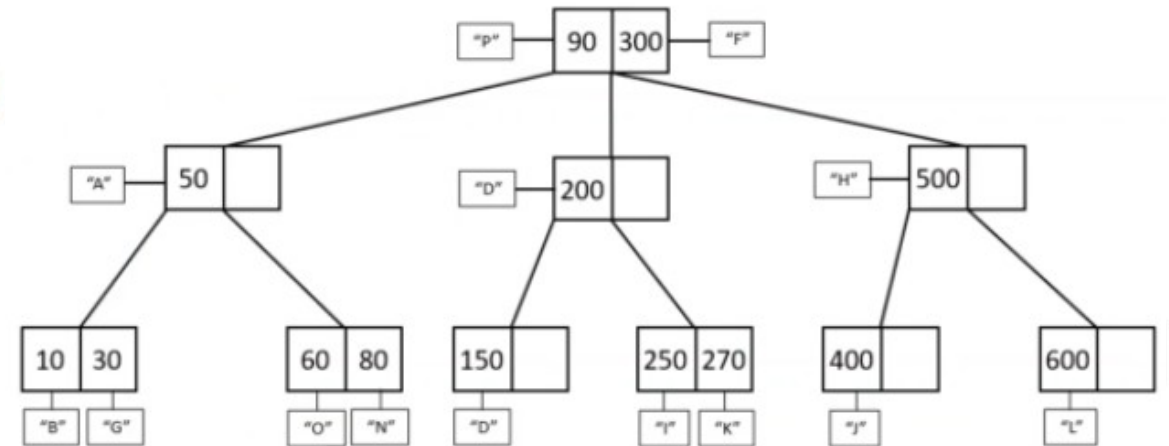
B-Tree

100, 20, 250, 600, 270, 60, 10, 300, 80, 150, 400, 30, 90, 200, 50, 500

- Operações de **remoção** numa 2-3 B-Tree ($1 \leq n^{\circ} \text{ chaves no nó} \leq 2$)



20



Substituir o valor do nó a remover pelo valor máximo da subárvore da esquerda (valor imediatamente anterior)

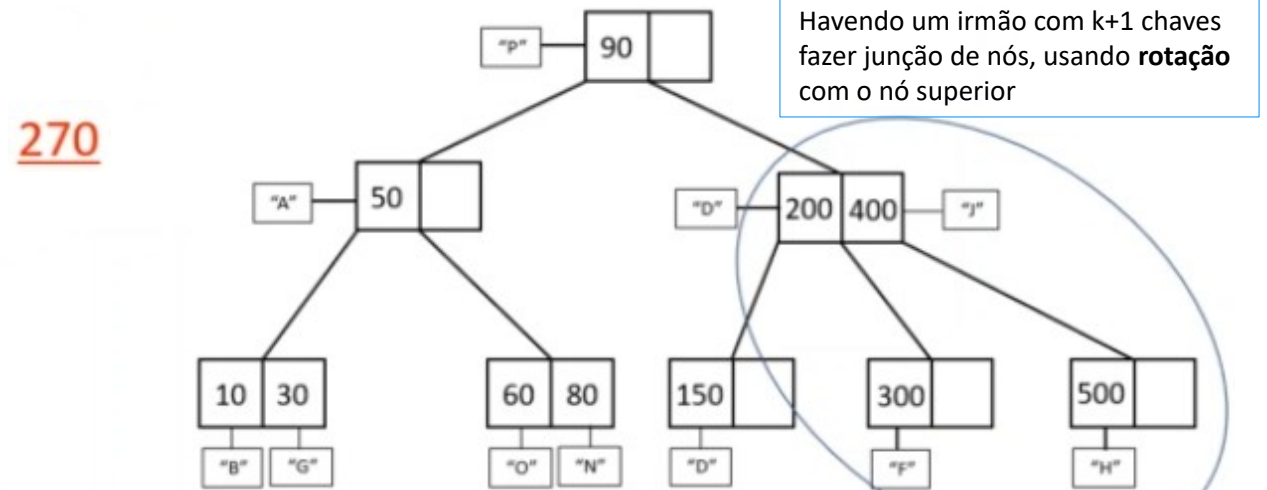
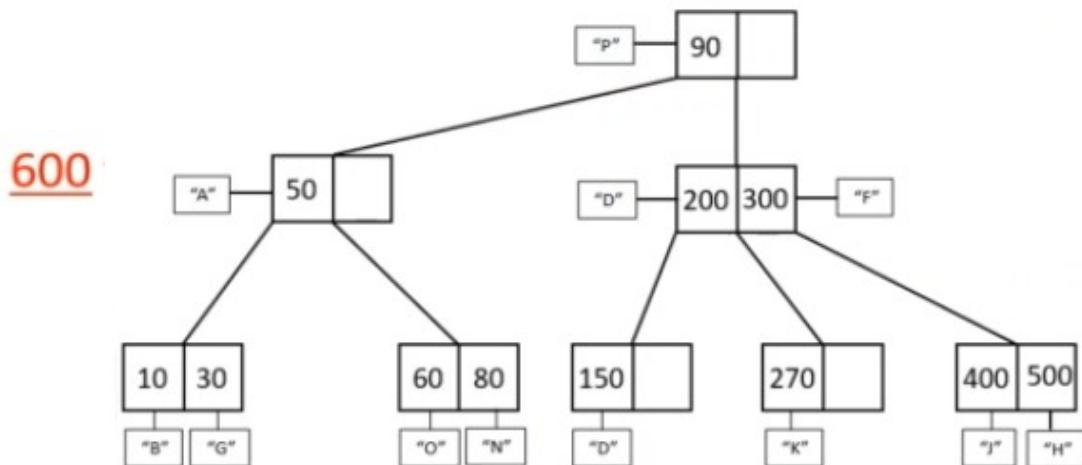
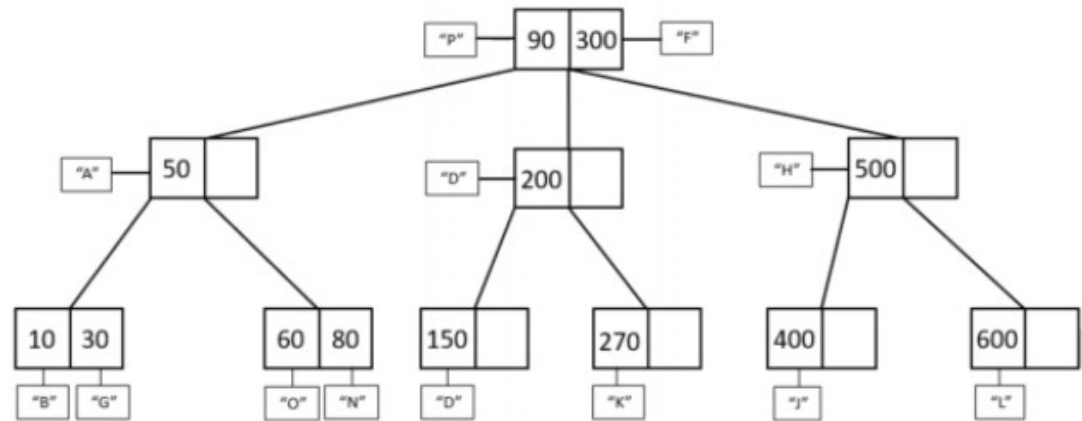
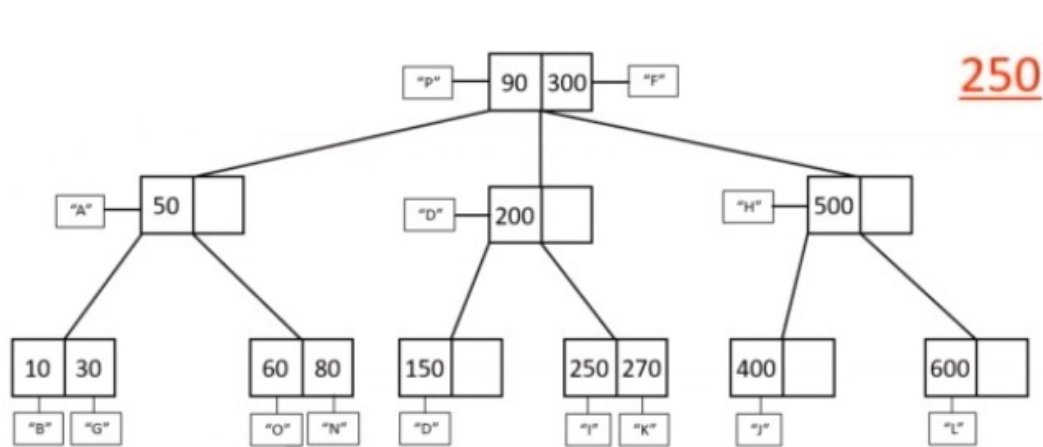
Como o nó ficou vazio encontrar um irmão com pelo menos $k + 1$ chaves para fazer **rotação**

Caso não haja procurar um irmão com k chaves para fazer **junção** de nós

B-Tree

100, 20, 250, 600, 270, 60, 10, 300, 80, 150, 400, 30, 90, 200, 50, 500

- Operações de **remoção** numa 2-3 B-Tree ($1 \leq n^{\circ} \text{ chaves no nó} \leq 2$)

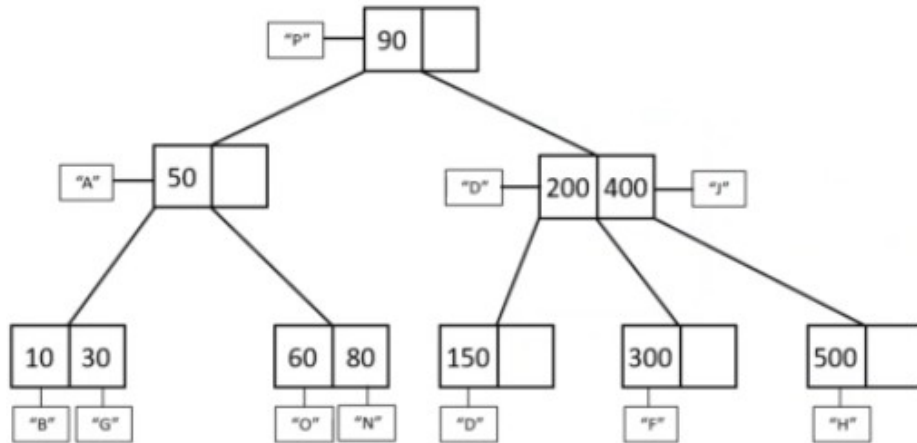


Havendo um irmão com $k+1$ chaves
fazer junção de nós, usando **rotação**
com o nó superior

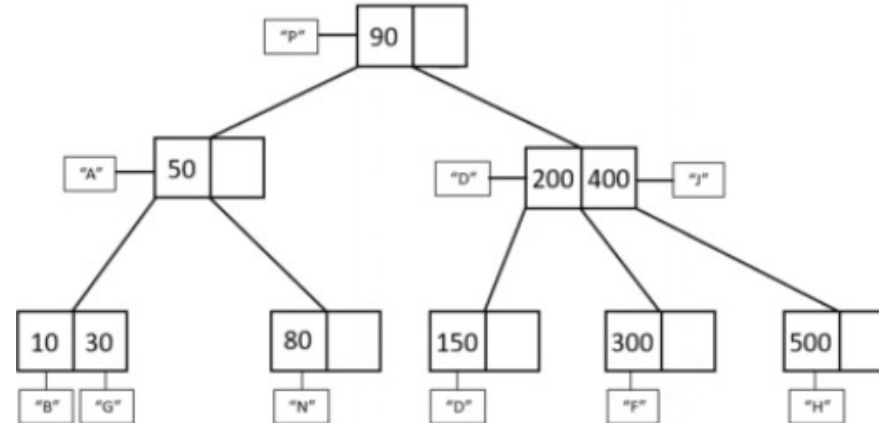
B-Tree

100, 20, 250, 600, 270, 60, 10, 300, 80, 150, 400, 30, 90, 200, 50, 500

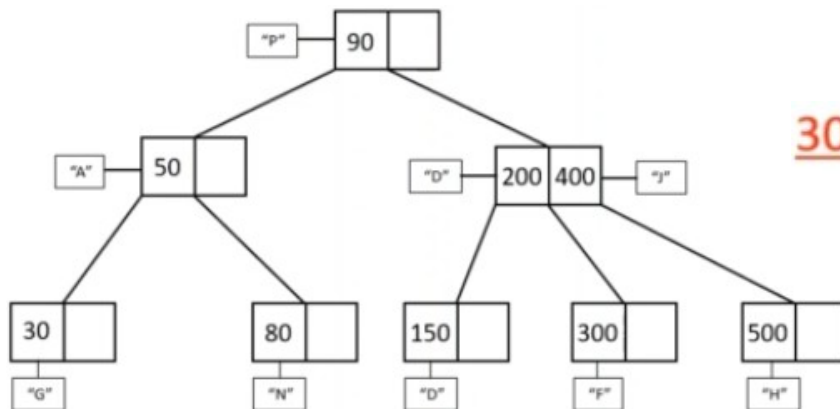
- Operações de **remoção** numa 2-3 B-Tree ($1 \leq n^o \text{ chaves no nó} \leq 2$)



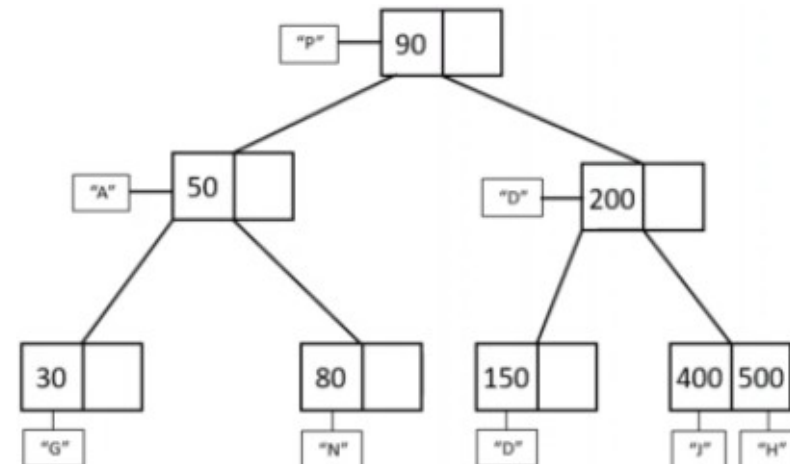
60



10



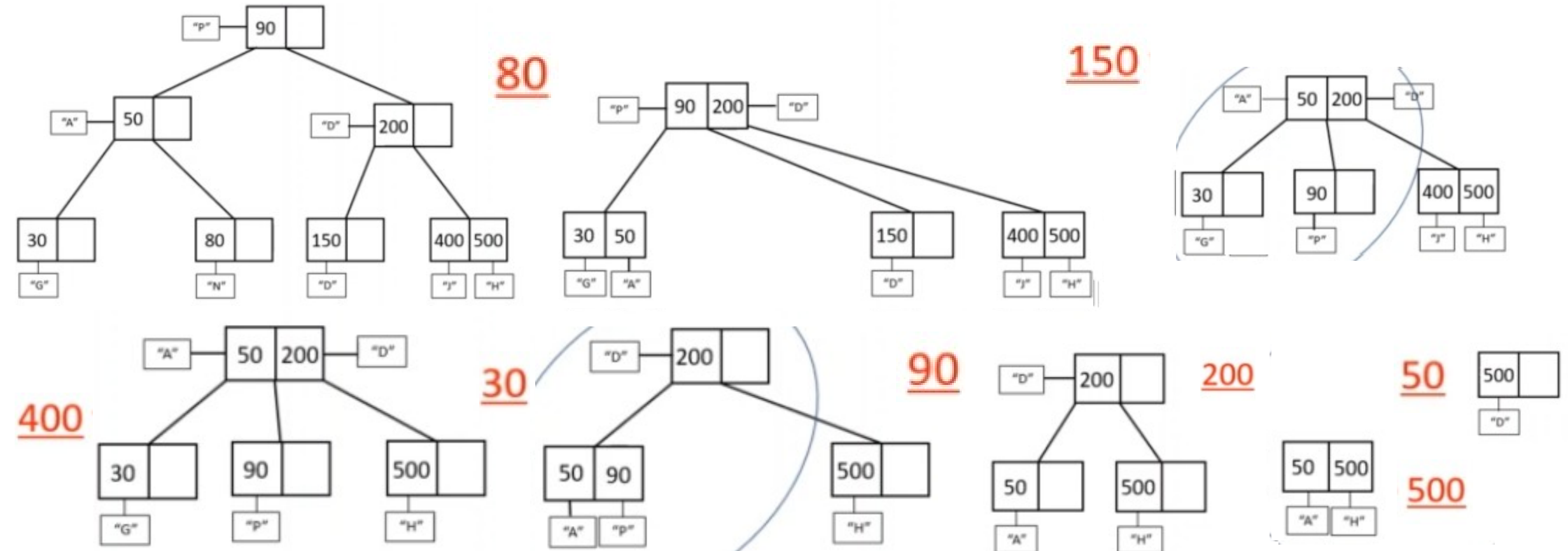
300



B-Tree

100, 20, 250, 600, 270, 60, 10, 300, 80, 150, 400, 30, 90, 200, 50, 500

- Operações de **remoção** numa 2-3 B-Tree ($1 \leq n^\circ \text{ chaves no nó} \leq 2$)



Objetivos de aprendizagem

- Fornecer conhecimentos para compreender:
 - estruturas de dados em árvores

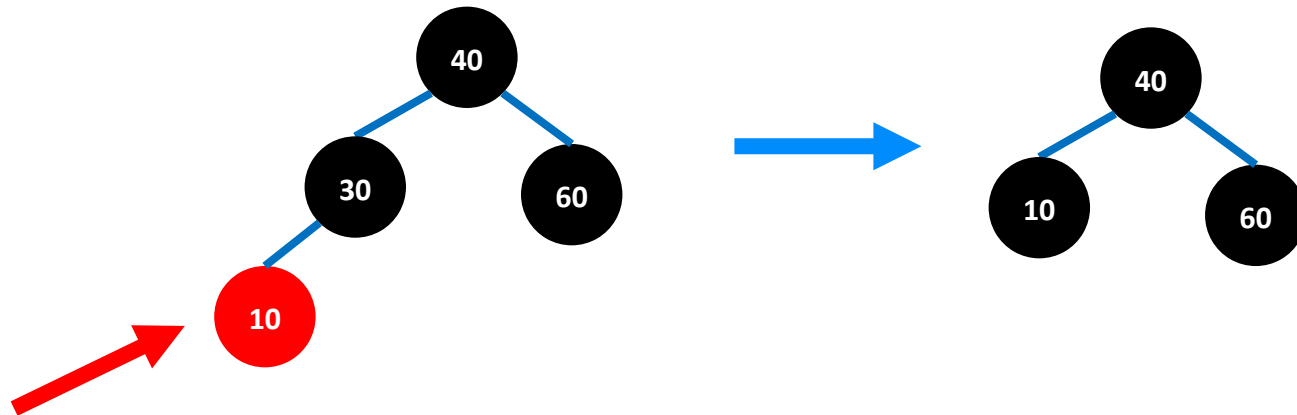
Anexos

Red Black Tree

- Regras gerais de balanceamento na remoção
 - Manter sempre um número igual de **black** nós no caminho para as folhas. O balanceamento é efetuado pela troca de cores e/ou rotação
 - Os nós internos não são removidos, sendo o seu valor alterado. Apenas os nós folha podem ser removidos
 - Só um nó folha **red** pode ser simplesmente removido.
 - Se um nó interno **black** for substituído por um nó folha **black**, este será um nó **duplo black**.
 - Se a raiz for um nó **duplo black** pode ser considerada apenas como nó **black**

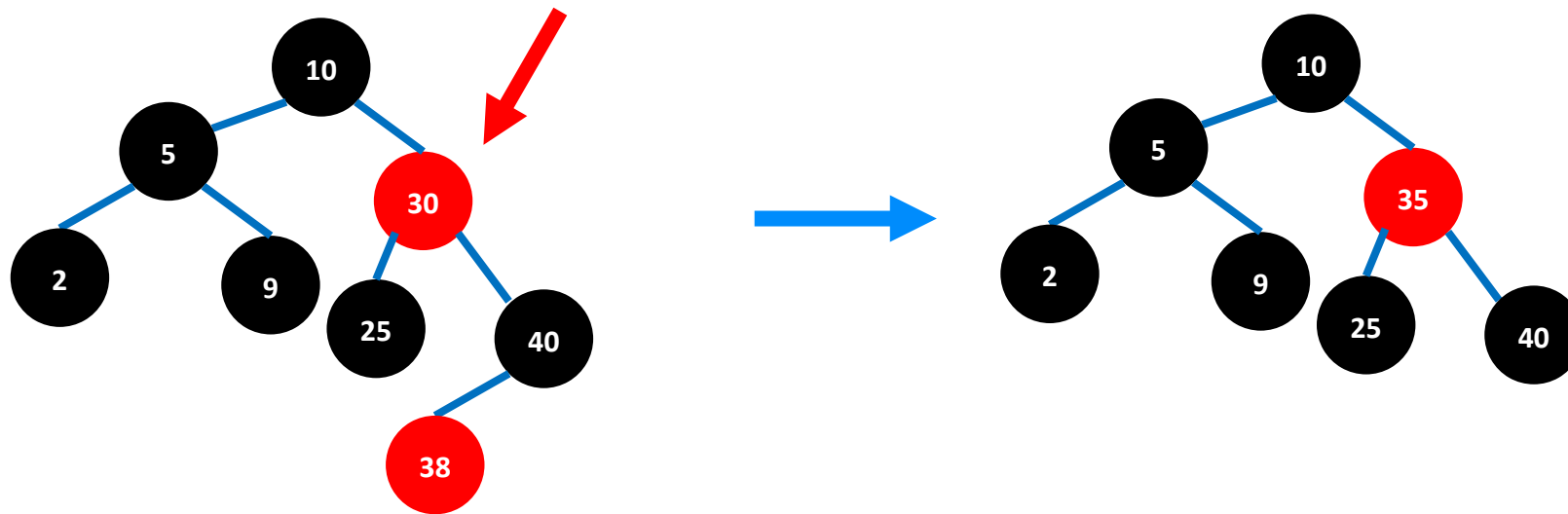
Red Black Tree

- Regras específicas de balanceamento na remoção
 - Caso 1a: remoção de um nó interior **black** com um nó filho **red** → substituir o valor do nó **black** pelo valor do nó **red** filho



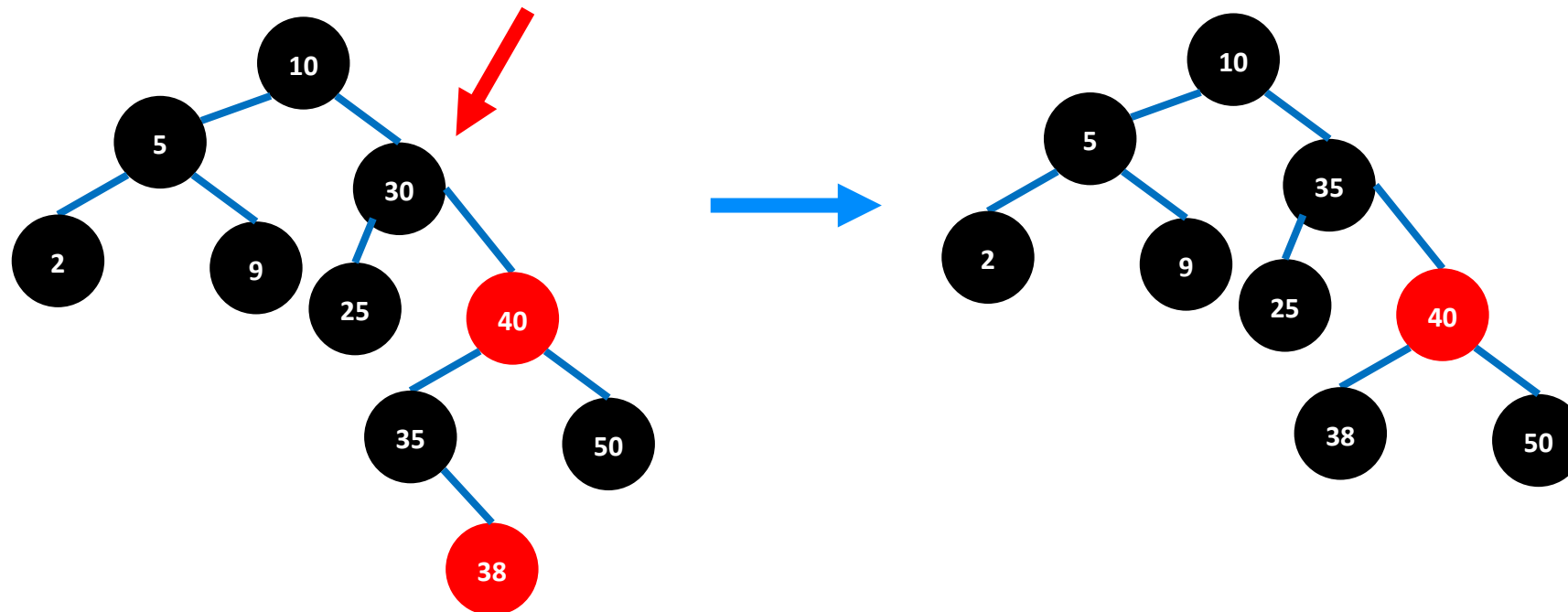
Red Black Tree

- Regras específicas de balanceamento na remoção
 - Caso 1b: remoção de um nó interior **red** com um nó folha **red** → substituir o valor do nó **red** pelo mínimo da subárvore à direita e remover o nó **red** mínimo/folha



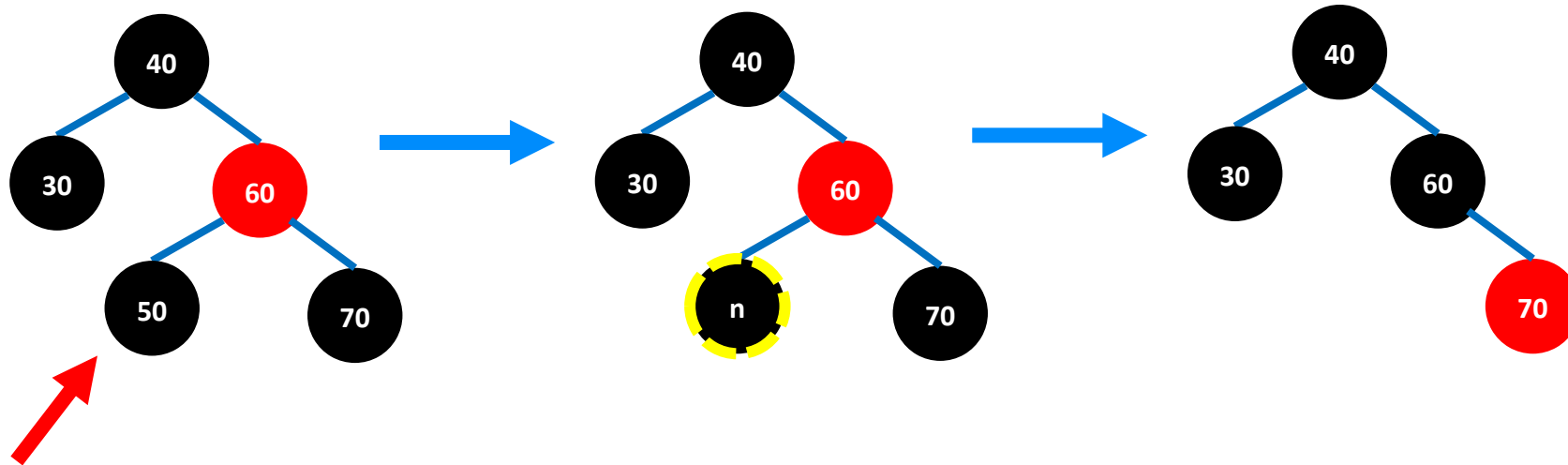
Red Black Tree

- Regras específicas de balanceamento na remoção
 - Caso 1c: remoção de um nó interior **black** com um nó folha **red** → substituir o valor do nó **black** pelo mínimo da subárvore à direita e o valor desse nó mínimo pelo do nó **red** folha



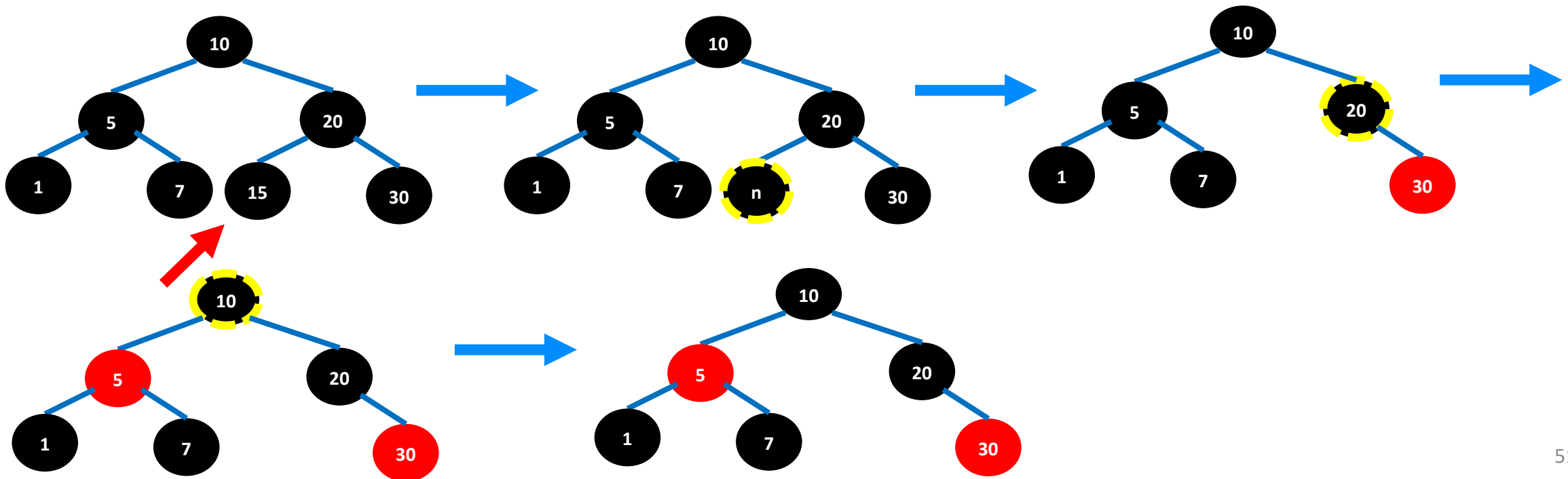
Red Black Tree

- Regras específicas de balanceamento na remoção
 - Caso 2a: remoção de nó filho **black** com pai **red**, e irmão **black** (com ambos filhos **black**) → o nó passa a **duplo black** e é removido; o nó pai passa a **black** e o irmão a **red**



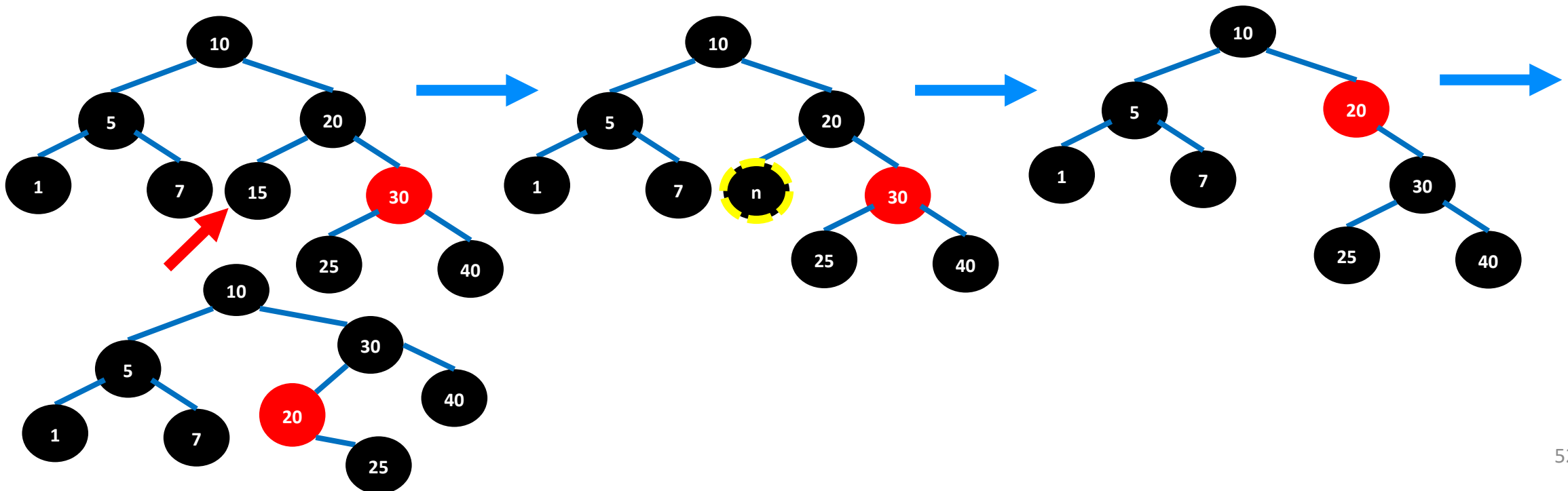
Red Black Tree

- Regras específicas de balanceamento na remoção
 - Caso 2b: remoção de nó filho **black** com pai **black**, e irmão **black** (com ambos filhos **black**) → o nó passa a **duplo black** e é removido; o nó pai passa **black** e o irmão passa a **red** (a transmissão do duplo **black** é efetuada recursivamente até à raiz)



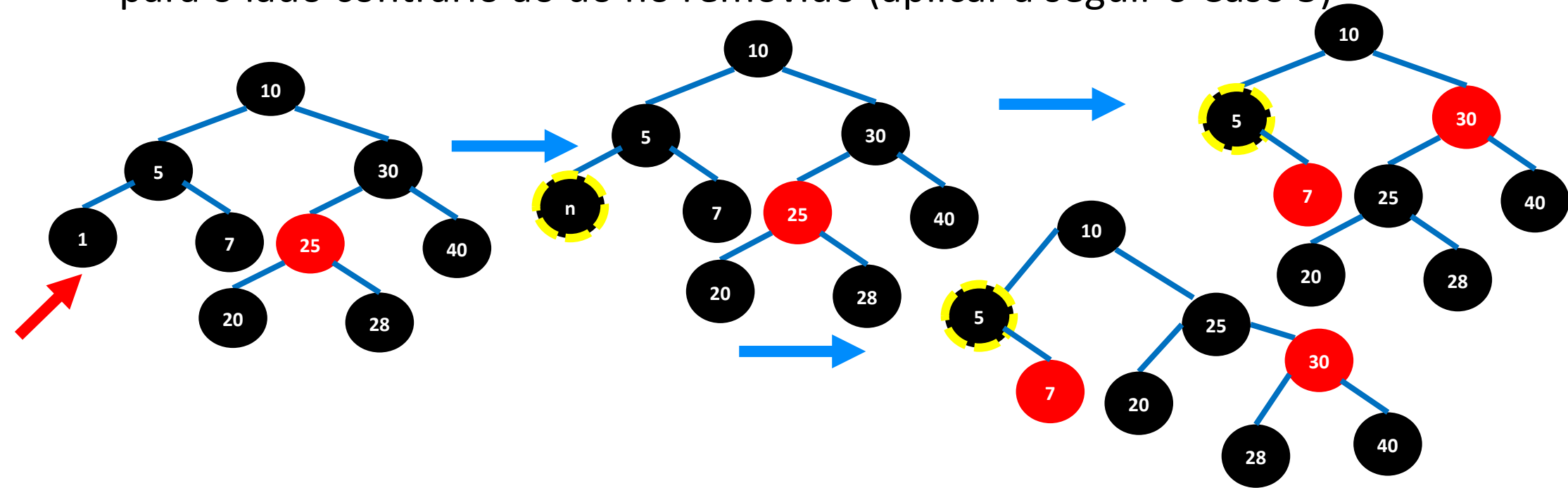
Red Black Tree

- Regras específicas de balanceamento na remoção
 - Caso 3: remoção de nó filho **black** com pai **black**, e irmão **red** (com ambos filhos **black**) → o nó passa a **duplo black** e é removido; o nó pai passa **red** e o irmão passa a **black** → rotação do nó pai para o lado do nó removido



Red Black Tree

- Regras específicas de balanceamento na remoção
 - Caso 4: remoção de nó filho **black** com pai **black**, e irmão **black**; primo mais afastado **black** e primo mais perto **red** → o nó passa a **duplo black** e é removido; o nó pai a **duplo black** e o irmão a **red**; o primo mais perto passa a **black** e o seu pai a **red** → rotação deste nó pai para o lado contrário ao do nó removido (aplicar a seguir o Caso 5)



Red Black Tree

- Regras específicas de balanceamento na remoção
 - Caso 5: rotação do nó raiz para o lado do nó removido (a transmissão do duplo **black** é recursiva até à raiz)

